

Seguridad Computacional

De Código a Release: Seguridad en Cada Etapa del SDLC

Curso IIC2531

Material compilado de clases sobre:

- Introducción a Seguridad
- Buffer Overflows y Baggy Bounds
- Autenticación de Usuarios
- Arquitectura de Seguridad (Google)
- OKWS y Separación de Privilegios
- Linux Containers

Incluye charla invitada de:
Carlos Oliva - Security Architect @ ZeroFox

30 de septiembre de 2025

Índice

I	Fundamentos de Seguridad Computacional	6
1.	Introducción a la Seguridad	6
1.1.	¿Qué es IIC2531?	6
1.2.	¿Por qué construir sistemas seguros es difícil?	6
1.3.	Plan de Alto Nivel para Pensar sobre Seguridad	6
1.4.	Lo que sale mal: Problemas con la política	7
1.4.1.	Ejemplo 1: Tarifa aérea de clase ejecutiva	7
1.4.2.	Ejemplo 2: Sistema escolar del condado de Fairfax, VA	7
1.4.3.	Ejemplo 3: Cuentas de Mat Honan	8
1.4.4.	Ejemplo 4: Cuenta de email de Sarah Palin	8
1.5.	Lo que sale mal: Problemas con el modelo de amenaza	8
1.5.1.	Asumir que el diseño/implementación es secreto	8
1.5.2.	Asumir que la mayoría de usuarios piensa en seguridad	8
1.5.3.	Asumir que usuarios no darán códigos 2FA al adversario	9
1.5.4.	Asumir un tipo particular de solución	9
1.5.5.	Asumir que está ejecutando el software esperado	9
1.5.6.	Suposiciones computacionales cambian con el tiempo	9
1.6.	Lo que sale mal: Problemas con el mecanismo (BUGS)	9
1.6.1.	Ejemplo 1: Límites de adivinación de contraseña de iCloud	10
1.6.2.	Ejemplo 2: Citigroup	10
1.6.3.	Ejemplo 3: Mala aleatoriedad para criptografía	10
1.7.	¿Cuál es el punto si no podemos lograr seguridad perfecta?	11
2.	Buffer Overflows y Defenses (Baggy Bounds)	11
2.1.	Recordatorios de C	11
2.1.1.	Punteros y Referencias	11
2.1.2.	Paso por Valor vs Paso por Referencia	12
2.2.	Manejo de Memoria	12
2.3.	Ejemplo: Buffer Overflow	12
2.4.	¿Por qué se cayó?	13
2.5.	¿Es esto un problema serio?	14
2.6.	Defensa 1: Stack No Ejecutable (NX bit)	14
2.7.	Defensa 2: Stack Canaries	14
2.8.	Defensa 3: Address Space Layout Randomization (ASLR)	15
2.9.	¿Ya terminamos?	15
2.10.	Buffer Overflows en Heap	15
2.11.	Enfoque de Verificación de Límites: Fat Pointers	16
2.12.	Baggy Bounds: Tabla Separada de Límites	17
2.13.	Truco de Potencia de Dos	17
2.14.	Limitaciones de Baggy Bounds	18
3.	Autenticación de Usuarios	18
3.1.	Un Problema Importante	18
3.2.	¿Dónde encaja la autenticación?	19
3.3.	Las Tres Partes de Autenticación	19
3.4.	Desafío: Principales Intermedios	19
3.5.	Desafío: ¿Cuál es la identidad del usuario?	20

3.6. Enfoques de Registro	20
3.7. Enfoques de Recuperación	20
3.8. Secreto Común: Contraseñas	20
3.9. Contraseñas: Difíciles por Factores Humanos	21
3.10. ¿Es buena idea cambiar contraseñas frecuentemente?	21
3.11. Defensa: Gestores de Contraseñas	21
3.12. Defender contra Adivinación	22
3.13. Limitar Intentos de Autenticación	22
3.14. Almacenar Contraseñas	22
3.14.1. Hashing	23
3.14.2. Salting	23
3.14.3. Hacer Hashing Costoso	23
3.15. Aumentar Contraseñas: Autenticación de Dos Factores	23
3.15.1. Variante 1: Código 2FA vía SMS	23
3.15.2. Variante 2: TOTP (Time-based One-Time Password)	24
3.15.3. Variante 3: U2F (Desafío-Respuesta)	24
3.16. El Protocolo U2F	25
3.17. Problema 1: Ataque de Phishing	25
3.18. Problema 2: Vinculación entre Sitios	26
3.19. Otros Ataques	26
4. Arquitectura de Seguridad: Google	27
4.1. ¿Qué es Arquitectura de Seguridad?	27
4.2. La Arquitectura de Seguridad Consiste en:	27
4.3. Caso de Estudio: Paper de Google	28
4.4. Entorno de Servidor de Google	28
4.5. Aislamiento: Punto de Partida para Seguridad	28
4.6. Máquinas Virtuales para Aislamiento	29
4.7. Compartir: Monitor de Referencia	29
4.8. Autenticar: ¿Quién/Qué emite la solicitud?	30
4.9. Autorizar: Determinar si Solicitud Debe Ser Permitida	30
4.10. Granularidad de Autorización	31
4.11. Ejemplos de Autorización en Google	31
4.12. ¿Por qué la estructura del Monitor de Referencia?	31
4.13. Defensa Perimetral vs Monitor de Referencia	31
4.14. Tickets de Permiso de Usuario Final	32
4.15. ¿Cómo Autentica un Servicio a Otro?	32
4.16. Criptografía y Gestión de Claves	32
4.17. Arranque Seguro de Servidores	33
4.18. Disponibilidad: Ataques DoS	33
5. OKWS: Separación de Privilegios	34
5.1. El Problema: ¿Qué Hacer con los Bugs?	34
5.2. Configuración Tradicional: Apache	35
5.3. Separación de Privilegios	36
5.4. Principios de OKWS	36
5.5. ¿Cómo Particiona OKWS el Servidor Web?	36
5.6. ¿Qué Daño si Cada Componente es Comprometido?	37
5.7. ¿Dónde Debería Buscar un Atacante Debilidades?	37
5.8. ¿Por Qué Proxies de Base de Datos Separados?	38
5.9. ¿Cómo Sabe dbproxy Qué Servicio Está Hablando?	38
5.10. Falta de Aislamiento por Usuario en OKWS	38

5.11. Mecanismos de Aislamiento en UNIX	39
5.12. ¿Cómo Hace Cumplir OKWS el Aislamiento?	39
5.13. ¿Por Qué okld, okd y Servicios Comparten chroot?	40
5.14. ¿Cómo Interactúan los Componentes OKWS?	40
5.15. Dificultades con Herramientas de Aislamiento UNIX	40
5.16. ¿Qué Ha Pasado Desde 2004?	41
6. Linux Containers	42
6.1. Recordatorio: Separación por Procesos	42
6.2. chroot	42
6.3. Casos de Uso de Contenedores	42
6.4. Máquinas Virtuales vs Containers	42
6.5. Linux Containers	42
6.6. Namespaces	43
6.7. Control Groups (cgroups)	43
6.8. Capabilities	44
6.8.1. Ejemplo: ping	44
6.8.2. Tipos de Capabilities	44
6.8.3. Ejemplo: Asignar Capability	44
6.9. seccomp-bpf	45
6.10. ¿Cómo Funciona Todo Junto?	45
II SDLC Seguro: De Código a Release	45
7. Hacia un SDLC Seguro	46
8. Vulnerabilidades Comunes en el SDLC	46
8.1. Herramientas DevOps e Infraestructura	46
8.2. Manipulación de Código	46
8.3. Prácticas Inseguras de Codificación	47
8.4. Filtraciones de Código y Datos Sensibles	47
9. Gobernanza de Seguridad en el SDLC	47
9.1. Visibilidad	47
9.2. Políticas de Privilegios Mínimos	47
9.3. Autenticación	48
9.4. Protección de Ramas y Reglas de Compilación	48
9.5. Supervisión de Cambios	48
10. Shift Left Security y DevSecOps	48
10.1. Conceptos Fundamentales	48
10.1.1. Shift Left Security	49
10.1.2. DevSecOps	49
11. Modelado de Amenazas	49
11.1. ¿Qué estamos construyendo?	49
11.2. ¿Qué puede salir mal?	50
11.3. ¿Qué haremos al respecto?	50
11.4. ¿Lo hicimos bien?	50

12.SLS: Planificación, Análisis y Diseño	51
12.1. Planificación	51
12.2. Análisis de Requerimientos	51
12.3. Diseño	51
13.DevSecOps: Codificación, Testing y Despliegue	52
13.1. Codificación	52
13.1.1. Static Application Security Testing (SAST)	52
13.1.2. Revisión y Aprobación de Código	52
13.1.3. Software Composition Analysis (SCA)	52
13.2. Testing y Despliegue	53
13.2.1. Dynamic Application Security Testing (DAST)	53
13.2.2. Seguridad en Contenedores	53
13.2.3. Ambientes Diferenciados	53
14.DevSecOps: Mantenimiento	54
14.1. Monitoreo	54
14.1.1. Herramientas de Telemetría	54
14.1.2. Web Application Firewall (WAF)	54
14.2. Reportería	54
14.3. Alertas	55
14.4. Bug Bounty	55
15.Protocolo de Respuesta a Incidentes	56
15.1. Equipos de Respuesta Especializados	56
15.2. Invocar Actores Involucrados	56
15.3. Identificar Severidad del Incidente	56
15.3.1. Superficie de Ataque y Exposición	56
15.3.2. Establecer Prioridad	56
15.3.3. CVSS (Common Vulnerability Scoring System)	57
15.4. Identificar Causa y Soluciones	57
15.5. Documentar y Comunicar	58
15.5.1. Template de Documento de Incidente	58
15.5.2. Comprometer Tareas y Asignar Responsabilidades	58
15.5.3. Comunicación Responsable	58
15.6. Reuniones de Equipos y Revisión	59
15.7. Post Mortem	59
16.Metodologías Populares para SSDLC	60
16.1. Microsoft Security Development Lifecycle (SDL)	60
16.2. OWASP Software Assurance Maturity Model (SAMM)	60
16.3. NIST Cybersecurity Framework (NIST CSF)	60
16.4. Google Supply-chain Levels for Software Artifacts (SLSA)	61
16.5. ISO/IEC 27001	61
17.Referencias y Recursos	62
17.1. OWASP Top 10	62
17.2. Calculadora CVSS	62
17.3. Recursos de Aprendizaje	63
17.3.1. Secure Developer Podcast	63
17.3.2. LiveOverflow en YouTube	63

18. Conclusiones	63
18.1. Principios Fundamentales	64
18.2. Cambio Cultural	64
18.3. Balance	64
18.4. Próximos Pasos	65

Parte I

Fundamentos de Seguridad Computacional

1 Introducción a la Seguridad

1.1 ¿Qué es IIC2531?

Curso general sobre Seguridad Computacional que busca amplitud más que profundidad en los temas. El contenido incluye:

- Threat Models (Modelos de Amenaza)
- Arquitectura Enfocada en Seguridad
- Separación de Privilegios
- Aislamiento de Componentes
- Seguridad en Dispositivos Móviles
- Seguridad en Redes

1.2 ¿Por qué construir sistemas seguros es difícil?

Ejemplo: archivo de notas de IIC2531 almacenado en un servidor.

Política: Solo los ayudantes deberían poder leer y escribir el archivo con las notas.

El problema:

- Fácil implementar el aspecto *positivo*: solo crear una URL que permita acceso
- Difícil garantizar el aspecto *negativo*: asegurar que NO haya forma complicada de que un no-ayudante acceda

Gran número de ataques potenciales:

- Explotar un error en el código del servidor
- Adivinar la contraseña de un ayudante
- Robar el laptop de un ayudante
- Interceptar las notas cuando se envían por internet
- Romper el esquema criptográfico usado
- Conseguir un trabajo en el DCC o como ayudante

1.3 Plan de Alto Nivel para Pensar sobre Seguridad

1. **Objetivo:** Lo que su sistema está tratando de lograr

- Ejemplo: solo Alice debería leer el archivo F
- Categorías: Confidencialidad, Integridad, Disponibilidad

2. **Modelo de Amenaza:** Suposiciones sobre lo que el atacante puede hacer

- Ejemplo: puede adivinar contraseñas, no puede robar físicamente nuestro servidor

3. **Política:** Reglas que harán que su sistema logre el objetivo

- Ejemplo: establecer permisos en F para que solo los procesos de Alice puedan leerlo
- Ejemplo: requerir contraseña y autenticación de dos factores

4. **Mecanismo:** Software/hardware que su sistema usa para hacer cumplir la política

- Ejemplo: cuentas de usuario, contraseñas, permisos de archivo, encriptación

BONUS

La Tríada CIA:

Los tres objetivos fundamentales de seguridad:

- **Confidencialidad (Confidentiality):** La información solo es accesible a quienes están autorizados
- **Integridad (Integrity):** La información y sistemas no pueden ser modificados de manera no autorizada
- **Disponibilidad (Availability):** La información y sistemas están disponibles cuando se necesitan

Diferentes sistemas priorizan diferentes elementos:

- Sistemas militares: Confidencialidad ¿Integridad ¿Disponibilidad
- Sistemas bancarios: Integridad ¿Disponibilidad ¿Confidencialidad
- Sistemas de emergencia: Disponibilidad ¿Integridad ¿Confidencialidad

1.4 Lo que sale mal: Problemas con la política

1.4.1 Ejemplo 1: Tarifa aérea de clase ejecutiva

Las aerolíneas permitían cambiar boletos de clase ejecutiva sin tarifas. En algunos sistemas, el boleto podía cambiarse incluso **después** del embarque.

Ataque: El adversario aborda el avión, cambia el boleto al siguiente vuelo, vuelve a abordar, ad infinitum.

Política revisada: El boleto no puede cambiarse una vez que el pasajero ha abordado.

1.4.2 Ejemplo 2: Sistema escolar del condado de Fairfax, VA

Políticas originales:

- El estudiante puede acceder solo a sus propios archivos
- El superintendente tiene acceso a los archivos de todos
- Los maestros pueden agregar nuevos estudiantes a su clase
- Los maestros pueden cambiar la contraseña de los estudiantes en su clase

Ataque:

1. Estudiante obtiene contraseña del maestro
2. Agrega al superintendente a la clase del maestro comprometido
3. Cambia la contraseña del superintendente
4. Inicia sesión como superintendente y obtiene acceso a todos los archivos

Lección: La política equivale a: los maestros pueden hacer cualquier cosa.

1.4.3 Ejemplo 3: Cuentas de Mat Honan

Cadena de ataques a través de múltiples servicios:

1. **Gmail:** Restablecimiento de contraseña envía enlace a dirección de backup (@me.com de Apple)
2. **Amazon:** Permitía agregar tarjetas de crédito sin autenticación
3. **Amazon:** Permitía cambiar email de cuenta con cualquier tarjeta registrada
4. **Amazon:** Mostraba últimos 4 dígitos de todas las tarjetas guardadas
5. **Apple:** Restablecimiento requería dirección de facturación + últimos 4 dígitos
6. **Resultado:** Acceso completo a Gmail

Lecciones:

- Los ataques juntan partes aparentemente no relacionadas
- Políticas individuales OK, pero la combinación no lo es
- Apple ve los últimos 4 dígitos como secreto, pero otros sitios no

1.4.4 Ejemplo 4: Cuenta de email de Sarah Palin

Yahoo permitía iniciar sesión con:

- Contraseña, O
- Respuesta a preguntas de seguridad

Adversario adivinó escuela secundaria, cumpleaños, etc.

Problema: No hay forma de hacer cumplir "Solo si el usuario olvida la contraseña"

Lección: El usuario debe asegurar que la contraseña *y* las preguntas de seguridad sean ambas difíciles de adivinar.

1.5 Lo que sale mal: Problemas con el modelo de amenaza

1.5.1 Asumir que el diseño/implementación es secreto

"Seguridad a través de la oscuridad"

Ejemplos de fallas:

- Chip Clipper
- Funciones criptográficas secretas rotas

1.5.2 Asumir que la mayoría de usuarios piensa en seguridad

- Usuario recibe email: "haga clic aquí para renovar su cuenta"
- Aparece cuadro de diálogo: "¿Realmente quiere instalar este programa?"
- Soporte técnico recibe llamada convincente para restablecer contraseña

1.5.3 Asumir que usuarios no darán códigos 2FA al adversario

La autenticación de dos factores defiende contra compromisos de contraseña, pero asume que el usuario mantendrá sus códigos secretos.

Realidad: El adversario puede engañar al usuario para que dé su código:

- Usuario no puede identificar sitio web legítimo vs adversario
- Especialmente problemático si el adversario pregunta por teléfono

1.5.4 Asumir un tipo particular de solución

Ejemplo: CAPTCHAs

Objetivo: Verificar si un humano se está registrando (prevenir registro masivo para spam)

Suposición original: El adversario tratará de construir OCR para resolver puzzles

Realidad: Solucionadores humanos de CAPTCHA en países del tercer mundo

- Costo: Fracción de un centavo por CAPTCHA
- Humanos son mejores que OCR

1.5.5 Asumir que está ejecutando el software esperado

Ejemplo: XcodeGhost

- Herramientas Xcode de Apple son grandes
- Descargarlas desde China a servidores de Apple era lento
- Espejos no oficiales dentro de China
- Algunos espejos contenían versión modificada que inyectaba malware
- Encontrado en varias aplicaciones iOS populares

1.5.6 Suposiciones computacionales cambian con el tiempo

Ejemplo: Kerberos del MIT

- Usó claves DES de 56 bits desde mediados de los 80
- Parecía razonable: adversario no puede verificar todas las 2^{56} claves
- Ahora: cuesta alrededor de \$100

1.6 Lo que sale mal: Problemas con el mecanismo (BUGS)

Los errores rutinariamente socavan la seguridad:

- Regla general: un error por 1000 líneas de código
- Errores en implementación de política de seguridad
- Errores en código aparentemente no relacionado con seguridad

Mentalidad correcta: Cualquier error es una explotación de seguridad potencial.

1.6.1 Ejemplo 1: Límites de adivinación de contraseña de iCloud

- Contraseñas débiles se pueden adivinar con pocos intentos (1K-1M)
- La mayoría de servicios limitan tasa de intentos de login
- iCloud tenía muchas APIs
- Una API (servicio "Find my iPhone") olvidó implementar limitación de tasa
- Atacante podía hacer millones de adivinaciones/día

Lección: Si se requieren muchas verificaciones, una faltará.

1.6.2 Ejemplo 2: Citigroup

- Usuarios de tarjetas de crédito accedían a cuentas en línea
- URL incluía números relacionados con número de cuenta: `x.citi.com/id=1234`
- Adversario probó diferentes números
- **El servidor no verificó que estuviera logueado en esa cuenta**
- Obtuvo información de cuenta de diferentes personas

Lección: Los programadores tienden a pensar solo en operación intencionada.

1.6.3 Ejemplo 3: Mala aleatoriedad para criptografía

Debian y OpenSSL:

- Aleatoriedad se inicializó usando código C no estrictamente correcto
- Herramienta de análisis marcó como problema
- Desarrolladores removieron las líneas "ofensivas"
- Esto previno sembrar el PRNG (Pseudo-Random Number Generator)
- API retornaba números "aleatorios" pero adivinables
- Adversario puede adivinar claves, impersonar servidores

Android y Bitcoin:

- Sistema olvidó sembrar SecureRandom
- Algunas claves de Bitcoin resultaron fáciles de adivinar
- Adversarios buscaron claves adivinables, gastaron bitcoins
- Realmente el nonce en firma ECDSA no era aleatorio
- Nonce repetido permite deducir clave privada

1.7 ¿Cuál es el punto si no podemos lograr seguridad perfecta?

- Raramente se requiere seguridad perfecta
- Hacer que costo del ataque > valor de la información
- Hacer nuestros sistemas menos atractivos que otros
- Encontrar técnicas con gran recompensa de seguridad
- A veces la seguridad aumenta el valor para el defensor:
 - VPNs dan flexibilidad para trabajar en casa
 - Sandboxing permite ejecutar software no confiable
- Tampoco hay seguridad física perfecta
- Gran diferencia: ataques digitales son baratos

BONUS

El Principio de Kerckhoffs:

En criptografía, un sistema debe ser seguro incluso si todo sobre el sistema, excepto la clave, es conocimiento público.

Reformulación moderna (Principio de Shannon): "El enemigo conoce el sistema"

Implicaciones para seguridad:

- No confiar en "security through obscurity"
- Asumir que el código fuente es público
- Diseños deben ser seguros bajo escrutinio
- La seguridad debe residir en las claves/secreto, no en el algoritmo

Ventajas del código abierto en seguridad:

- "Many eyes make bugs shallow" (Ley de Linus)
- Auditoría pública
- Imposible ocultar backdoors
- Confianza verificable

Contraargumento: Código abierto también ayuda a atacantes. Sin embargo, la historia muestra que los sistemas más seguros (OpenSSL, Linux kernel security modules) son open source.

2 Buffer Overflows y Defenses (Baggy Bounds)

2.1 Recordatorios de C

2.1.1 Punteros y Referencias

```
1 int a;           // variable entera
2 &a              // dirección de a
3 int *a = malloc(sizeof(int)); // puntero a entero
4 *a             // valor apuntado por a
```

2.1.2 Paso por Valor vs Paso por Referencia

Paso por valor (no modifica original):

```

1 void modifyValue(int x) {
2     x = 100;
3 }
4
5 int main() {
6     int x = 50;
7     printf("Before: %d\n", x); // Output: 50
8     modifyValue(x);
9     printf("After: %d\n", x);  // Output: 50
10    return 0;
11 }
```

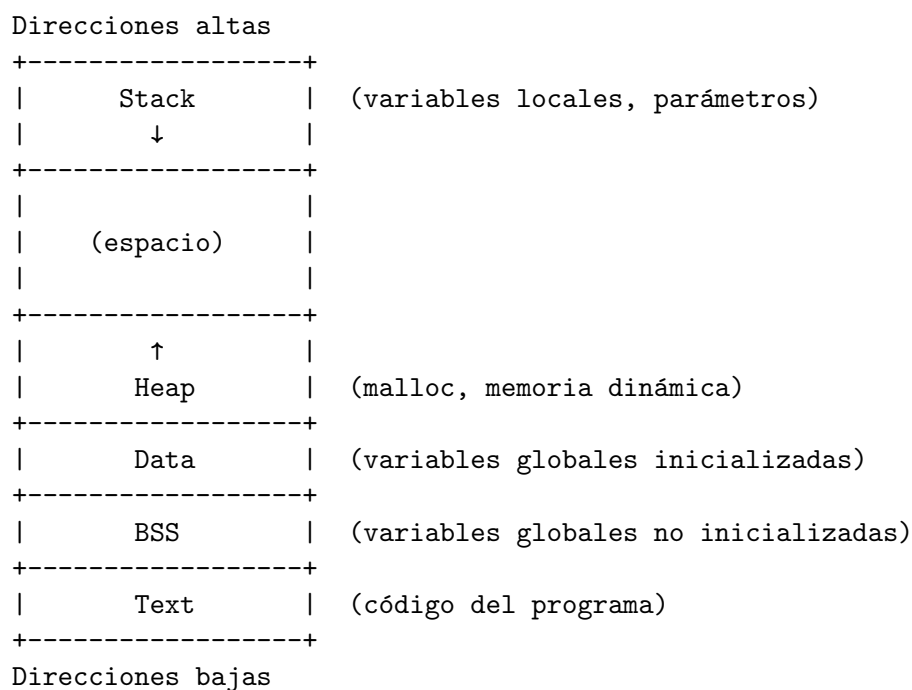
Paso por referencia (modifica original):

```

1 void modifyValue(int *ptr_to_x) {
2     *ptr_to_x = 100;
3 }
4
5 int main() {
6     int x = 50;
7     printf("Before: %d\n", x); // Output: 50
8     modifyValue(&x);
9     printf("After: %d\n", x);  // Output: 100
10    return 0;
11 }
```

2.2 Manejo de Memoria

El layout de memoria de un programa en ejecución:



2.3 Ejemplo: Buffer Overflow

```

1  #include <stdio.h>
2  #include <stdlib.h>
3
4  char *gets(char *buf) {
5      int c;
6      while((c = getchar()) != EOF && c != '\n')
7          *buf++ = c;
8      *buf = '\0';
9      return buf;
10 }
11
12 int read_req(void) {
13     char buf[128];
14     int i;
15     gets(buf);
16     i = atoi(buf);
17     return i;
18 }
19
20 int main() {
21     int x = read_req();
22     printf("x=%d\n", x);
23 }

```

Compilación:

```
gcc -g -static -fno-stack-protector -fcf-protection=none readreq.c -o readreq
```

Ejecución:

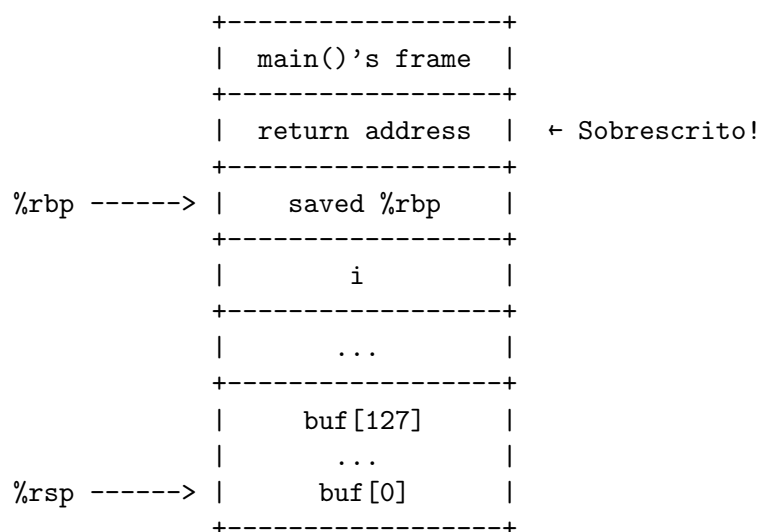
```

./readreq
1234      # Funciona bien
./readreq
AAAAAAA...A # Segmentation fault

```

2.4 ¿Por qué se cayó?

Stack de read_req():



Si escribimos 190 'A's:

- Sobrescribimos buf[128]
- Sobrescribimos saved %rbp
- Sobrescribimos return address con 0x41414141 ('AAAA')
- CPU intenta saltar a 0x41414141
- Segmentation fault

2.5 ¿Es esto un problema serio?

Sí. Ataque de "inyección de código":

1. Atacante coloca código malicioso en buf[]
2. Sobrescribe return address para apuntar a buf[]
3. Cuando función retorna, ejecuta código del atacante
4. El código puede hacer cualquier cosa que la aplicación pueda hacer

¿Qué puede hacer el adversario?

- Si proceso corre como root: cualquier cosa
- Incluso sin root: enviar spam, leer archivos, cargar programa desde red

2.6 Defensa 1: Stack No Ejecutable (NX bit)

El OS le dice al hardware que no ejecute en el stack:

- Intel permite establecer bit NX en cada página del stack
- Previene ejecución de instrucciones inyectadas

¿NX significa que no hay que preocuparse por buffer overflows?

- No - existen ataques más sofisticados (ROP - Return Oriented Programming)

¿NX es pérdida de tiempo si no es perfecto?

- Tampoco - eleva significativamente la barrera para atacantes

2.7 Defensa 2: Stack Canaries

Detecta modificación del PC de retorno *antes* de que sea usado por RET:

	+-----+	
entry %esp->	return address	^
	+-----+	
new %ebp -->	saved %rbp	
	+-----+	
	CANARY	Overflow
	+-----+	detectado
	buf [127]	aquí
	...	
	buf [0]	
	+-----+	

Funcionamiento:

1. Compilador genera código que empuja valor canario.^{a)}l entrar función
2. Antes del retorno, verifica que canario no haya cambiado
3. Si cambió, aborta el programa

¿Qué valor usar para canario?

- Número aleatorio elegido al inicio del programa
- Almacenado en ubicación protegida

Ejemplos: StackGuard, Stack Smashing Protector de gcc

2.8 Defensa 3: Address Space Layout Randomization (ASLR)

Colocar memoria del proceso en direcciones aleatorias:

- Stack en dirección aleatoria
- Heap en dirección aleatoria
- Código en dirección aleatoria
- Bibliotecas en direcciones aleatorias

Ventaja: El adversario no conoce dirección precisa para inyectar código

Limitación: Requiere soporte del compilador para código reubicable

2.9 ¿Ya terminamos?

No. Ataques que funcionan a pesar de estas defensas:

- Sobrescribir puntero de función antes del canary
- Sobrescribir variable crucial en stack:

```
1 bool ok = ...;
2 char buf[128];
3 gets(buf);
4 if(ok) { ... }
```

- Desbordamiento de variable global
- Desbordamiento de búfer en heap

2.10 Buffer Overflows en Heap

```
1 foo(){
2     char *p = malloc(16);
3     gets(p);
4 }
```

¿Puede el atacante predecir qué está después de p?

Algunos malloc() organizan bloques en lista doblemente enlazada:


```

^
|  data
|-- next
    prev --|
|-> ----  |
|  data   |
|-- next  |
    prev --|-|
|-> ---- <-| |
|  data   |
|-- next  |
    prev <---

```

Ataque de desenlace:

Cuando se asigna bloque libre:

```

1 b = elegir bloque libre
2 b->next->prev = b->prev;
3 b->prev->next = b->next;

```

Si atacante desborda y modifica next y prev:

- Suponer `b->prev = x`, `b->next = y`
- `malloc()` efectivamente ejecuta: `*y = x`
- ¡Escritura de valor elegido a cualquier ubicación!

2.11 Enfoque de Verificación de Límites: Fat Pointers

Cada puntero contiene inicio, fin y valor actual:

```

+-----+-----+-----+
| 32-bit start | 32-bit end | 32-bit curr |
+-----+-----+-----+

```

Compilador genera código para:

- Establecer start y end en `malloc()` y `p = &x`
- Durante desreferencia, verificar `start <= curr < end`
- Durante asignación de puntero, copiar todo el puntero gordo

Ejemplo:

```

1 p = malloc(4); // malloc llena start y end
2 *p = 1;        // verifica: start <= curr < end (OK)
3 q = p;         // copia puntero gordo a q
4 q = q + 8;     // suma 8 a q.curr
5 *q = 1;        // verifica: falla (curr >= end)

```

Problemas:

1. Verificaciones pueden ser costosas
2. No muy compatible:
 - No se puede pasar puntero gordo a biblioteca no modificada
 - Cambia tamaño de estructuras
 - Actualizaciones no son atómicas

2.12 Baggy Bounds: Tabla Separada de Límites

Mantener información de límites en tabla separada para no cambiar representación del puntero.

Idea básica:

- Para cada objeto asignado, almacenar inicio y tamaño
- malloc() o código generado mantiene tabla
- Establecer bit OOB (Out Of Bounds) si puntero sale de objeto

Para cada puntero, compilador genera código para:

1. **Aritmética de punteros:** `char *q = p + 256;`
 - Detectar si puntero ya no apunta al objeto original
 - Establecer bit OOB si es necesario
 - Comparar q con tabla[p]
2. **Desreferencia:** `char ch = *q;`
 - Hacer pánico si bit OOB de q está establecido

Desafíos:

- Indexar tabla[] con p y también con p + 10
- Si aritmética modifica p a objeto diferente, usar bit OOB
- Si aritmética vuelve p a objeto original, limpiar OOB
- Prevenir que tabla sea enorme o costosa

2.13 Truco de Potencia de Dos

Baggy Bounds optimiza con restricciones:

- Entrada de tabla mantiene solo tamaño como log base 2 (1 byte)
- $2^{\text{tabla}[i]}$ produce tamaño (ej. 20 = 1 megabyte)
- Asignar solo tamaños potencia de dos en límites potencia de dos
- Código puede calcular inicio desde puntero y tabla[]
- Cada entrada cubre slot_size=16 bytes
- Objeto de 64 bytes usa 4 entradas empezando en tabla[x/16]

Encontrar tamaño e inicio:

```

1 // Para slot_size=16:
2 tamano = 1 << tabla[p >> 4]; // 4 bits en 16
3 inicio = p & ~(tamano - 1); // limpia bits bajos

```

Truco de memoria virtual para OOB:

- Establecer bit más significativo en puntero OOB
- Marcar páginas en mitad superior del espacio como inaccesibles

- ¡No necesita instrucciones de verificación para desreferencias!
- OOB significa "ptr dentro de $\text{slot_size}/2$ del objeto original"

Ejemplo:

```

1 p = malloc(25); // redondea a 32, establece tabla[]
2 q = p + 10;    // OK
3 r = p + 35;    // establece OOB
4 *r = 1;        // sistema VM fuerza crash
5 s = p + 35;    // establece OOB
6 s = s - 20;    // limpia OOB
7 *s = 1;        // OK
8 t = p + 100;   // crash: no dentro de slot_size/2

```

2.14 Limitaciones de Baggy Bounds

¿Qué errores podría NO detectar?

- Buffer overflow de array dentro de struct malloc()'d más grande
- Convertir puntero a int, modificar, convertir de vuelta
- Aplicación implementa su propio asignador
- Punteros colgantes a memoria re-asignada

BONUS

Evolución de defensas contra buffer overflows:

1. **Años 90:** Stack canaries (StackGuard)
2. **Años 2000:** DEP/NX bit
3. **Años 2000s:** ASLR
4. **Años 2010:** Control Flow Integrity (CFI)
5. **Años 2010s:** Memory tagging (ARM MTE)
6. **Actualidad:** Migración a lenguajes memory-safe (Rust, Go)

Estado actual:

- Buffer overflows aún causan 70 % de vulnerabilidades en código C/C++
- Industria moviéndose hacia lenguajes seguros:
 - Android: Rust para componentes nuevos
 - Linux kernel: Soporte de Rust desde 6.1
 - Microsoft: C# para nuevos proyectos, Rust para infraestructura
- Código legacy C/C++ seguirá por décadas
- Defensas en profundidad siguen siendo esenciales

3 Autenticación de Usuarios

3.1 Un Problema Importante

La autenticación es:

- Base de muchas políticas de seguridad
- Presenta problemas técnicos interesantes
- Fácil de hacer mal
- Continúa siendo desafío porque seguridad no es solo técnica

3.2 ¿Dónde encaja la autenticación?

Modelo de guardia de seguridad:

cliente -> solicitud -> [GUARDIA] -> servidor/recurso

- Servidor contiene recurso nombrado por solicitud
- Guardia verifica cada solicitud
- **Mediación completa:** todas solicitudes verificadas
 1. Aislamiento del servidor
 2. Guardia invocado en todas las solicitudes
- **Desafío:** ¿Cómo autentica el guardia una solicitud?

3.3 Las Tres Partes de Autenticación

1. Registro

- Establecer secreto entre usuario y guardia

2. Verificación

- Usuario envía (usuario, secreto) con solicitud
- Guardia verifica que secreto coincida

3. Recuperación

- Usuario pierde secreto
- A menudo pasado por alto - puede ser talón de Aquiles

3.4 Desafío: Principales Intermedios

Usuarios rara vez emiten solicitudes directamente:

Usuario -> Dispositivo -> Balanceador -> App Server -> Recurso

- Principales intermedios "hablan por" el usuario
- Importante considerar estas entidades como principales
- Necesitamos confiar en cada principal intermedio

Ejemplo: inicio de sesión en sitio web

- Solicitud viene del dispositivo del usuario
- Usuario escribe contraseña/código en dispositivo
- ¿Qué pasa si atacante controla dispositivo? (malware/keylogger)
- Atacante puede lanzar ataque de hombre en el medio (MITM)

3.5 Desafío: ¿Cuál es la identidad del usuario?

- Usuario registra secreto - ¿quién es realmente?
- A escala universitaria, podríamos verificar identidad al registrarse
- Típicamente nos conformamos con garantía más débil:
 - Usuario que inicia sesión tiene secreto de registro
 - Asumimos que es mismo usuario
 - No hay garantía de verdadera identidad
- Para muchos usos está bien (ej. Amazon solo importa que pagues)

3.6 Enfoques de Registro

1. Primero en llegar, primero en ser servido

- Ej: registrarse para cuenta en gmail.com

2. Arrancar desde otro mecanismo

- Ej: verificar vía email

3. Creado por administrador

- Ej: nuevo empleado en empresa

3.7 Enfoques de Recuperación

- "Preguntas de seguridad": política OR
- Verificar vía email
- Probar conocimiento de número de tarjeta de crédito
- Crear nueva cuenta (si no es importante retener mismo principal)
- Llamar servicio al cliente: salida de emergencia sin política precisa
 - A menudo susceptible a ataques de ingeniería social

3.8 Secreto Común: Contraseñas

Funcionamiento:

- Principal y guardia comparten conjunto secreto de bits
- Usuario escribe nombre de usuario y contraseña
- Guardia verifica si contraseña es correcta

Ventaja:

- Fácil de usar e implementar

Desventajas:

- Contraseñas son a menudo muy valiosas
- Factor humano → secretos débiles y compartidos

3.9 Contraseñas: Difíciles por Factores Humanos

1. Usuarios eligen contraseñas adivinables

- 20 % de cuentas usan mismo conjunto de 5,000 contraseñas populares
- No se puede permitir 5,000 intentos de adivinación
- No se puede permitir que adversario adivine "123456" para cada usuario

2. Contraseñas comunes contienen dígitos, mayúsculas, etc.

- ¿Es "1Password!" una buena contraseña?
- Lo que importa es la **entropía**
- Requisitos de caracteres no son especialmente útiles

3. Entropía de contraseña

- Expresada en bits
- Contraseña conocida: 0 bits
- Adivinada en primer intento 50 % del tiempo: 1 bit
- 16 bits de entropía: 2^{16} intentos para todas las posibilidades

4. Contraseñas compartidas entre sitios/aplicaciones

- Importante cuando hablamos de cómo usar y almacenar contraseñas

5. Queremos animar contraseñas de alta entropía

3.10 ¿Es buena idea cambiar contraseñas frecuentemente?

Depende de las amenazas.

Beneficios:

- Si adversario obtuvo contraseña antigua, ya no es útil
- Tal vez fuerza a usuario a no reutilizar entre sitios

Desventajas:

- Usuario podría tener dificultades para recordarla
- Usuario podría elegir contraseña más débil
- Usuario podría escribirla en algún lugar

No hay política clara ganadora.

3.11 Defensa: Gestores de Contraseñas

El problema:

- Usuarios tentados a usar contraseñas simples (pueden recordarlas, baja entropía)
- Usuarios tentados a usar misma contraseña para diferentes sitios (¡mala idea!)

Gestores de contraseñas: Contraseñas convenientes, fuertes y diferentes

- Gestor elige contraseña con alta entropía

- Gestor almacena diferentes contraseñas para diferentes sitios
- Opcional: gestor llena campo de contraseña automáticamente
- Usuario debe autenticarse al gestor (recordar una contraseña fuerte)
- **El gestor es confiable!**

Ejemplos: 1Password, LastPass, Bitwarden, KeePass

3.12 Defender contra Adivinación

El problema:

- Ataques de adivinación por pequeño espacio de claves
- Adversario tiene información sobre distribuciones de contraseñas
- Contraseñas comunes (filtraciones de bases de datos)
- Frases populares de sitios web
- Sesgos comunes en selección de caracteres

Datos encriptados con contraseña:

- Vulnerables a adivinación offline
- No hay servidor verificando intentos

3.13 Limitar Intentos de Autenticación

- No queremos permitir que adversario adivine contraseñas
- Importante limitar tasa de intentos de login
- Implementar períodos de timeout después de demasiados intentos incorrectos
- Limitar por usuario podría no ser suficiente
 - Adversario puede adivinar "123456" para cada nombre de usuario
- ¿CAPTCHAs?
 - Costo económico de resolver CAPTCHAs es bajo
- Mayoría de sistemas tienen varias heurísticas

3.14 Almacenar Contraseñas

Plan ingenuo: Tabla con (nombre de usuario, contraseña)

Riesgos:

1. Adversario que compromete servidor aprende todas las contraseñas
2. Incluso después de recuperación, debe restablecer todas las contraseñas
3. Adversario puede usar mismas contraseñas en otros servicios

3.14.1 Hashing

Almacenar pares de (nombre de usuario, $H(\text{contraseña})$)

- Se puede verificar hasheando contraseña suministrada
- Hash criptográfico es unidireccional (no se puede invertir)

3.14.2 Salting

Problema: Rainbow tables - diccionario de hashes de contraseñas comunes

Solución: Almacenar (nombre de usuario, salt, $H(\text{salt} \text{ — } \text{contraseña})$)

- Se verifica hasheando contraseña suministrada con salt conocido
- Ahora misma contraseña puede corresponder a muchos hashes diferentes
- Costoso construir tabla de todas las combinaciones salt+contraseña

3.14.3 Hacer Hashing Costoso

- Funciones de hash crypto típicas son rápidas
- Adversario no está limitado en tasa al adivinar contra lista comprometida
- **Solución:** Usar función de hash intencionalmente costosa
 - Llamada Key Derivation Function (KDF)
- Ejemplos: bcrypt, scrypt, PBKDF2, Argon2

3.15 Aumentar Contraseñas: Autenticación de Dos Factores

Ayuda a defender contra:

- Contraseñas débiles
- Reutilización de contraseñas
- Ataques MITM y phishing

3.15.1 Variante 1: Código 2FA vía SMS

Funcionamiento:

- Servidor almacena número de teléfono del usuario
- Servidor envía código al teléfono
- Usuario ingresa código en aplicación/sitio

Ventajas:

- Fácil de empezar a usar
- Fácil recuperarse de teléfono perdido
- Portabilidad de números entre proveedores

Desventajas:

- Confiar en red de telefonía celular y proveedor
- Requerir rango de red celular
- Ataques de phishing aún posibles

3.15.2 Variante 2: TOTP (Time-based One-Time Password)

Funcionamiento:

- Servidor y dispositivo acuerdan valor secreto (ej. escaneando QR)
- Dispositivo genera: código = $H(\text{secreto} \text{ — tiempo actual})$
- Servidor verifica que código corresponda al tiempo actual

Ventajas:

- No necesita red celular disponible
- No necesita confiar en proveedor celular

Desventajas:

- Setup implica instalar aplicación, cargar valor secreto
- Lidar con usuario cambiando dispositivos
- Aún susceptible a ataques de phishing

Aplicaciones: Google Authenticator, Authy, Microsoft Authenticator

3.15.3 Variante 3: U2F (Desafío-Respuesta)

Funcionamiento:

- Dongle USB del usuario tiene par de claves pública/secreta
- Servidor almacena clave pública del dongle
- Para login:
 1. Servidor envía cadena de desafío aleatoria a navegador
 2. Navegador envía desafío e identidad del servidor al dongle
 3. Dongle firma (desafío, identidad del servidor) con clave privada
 4. Servidor verifica firma

Ventajas:

- **No susceptible a phishing**
- No necesita configuración por servidor
- Compromiso del servidor no permite autenticación posterior

Desventajas:

- Necesita software especial en computadora
- Usuario necesita llevar dongle

3.16 El Protocolo U2F

Operaciones de cripto de clave pública:

- $\text{Sign}(\text{Kpriv}, m) \rightarrow \text{firma sig}$
- $\text{Verify}(\text{Kpub}, m, \text{sig}) \rightarrow \text{ok?}$

Estado:

- Dispositivo (D): (H/Origin, Kpub, Kpriv)
- Servidor (S): (H/Origin, Kpub)
- Navegador (B): ejecutando JavaScript de S
 - ¡Nunca ve Kpriv de D!
 - Incluso si B está comprometido, no puede robar Kpriv

Protocolo simplificado:

S -> B: desafío
B -> D: desafío
D -> B: $\text{Sign}(\text{Kpriv}, \text{desafío}) \rightarrow \text{firma s}$
B -> S: s
S: $\text{Verify}(\text{Kpub}, \text{desafío}, s) \rightarrow \text{ok?}$

Desafío = nonce (número aleatorio)

¿Por qué ataque de replay no funciona?

- Atacante graba s cuando víctima inicia sesión
- Más tarde atacante visita sitio, inicia sesión como víctima
- S elige nuevo desafío
- Atacante reenvía s grabado
- S calcula $\text{Verify}(\text{Kpub}, \text{nuevo desafío}, s)$, que falla
- Porque D firmó desafío antiguo, no el nuevo

3.17 Problema 1: Ataque de Phishing

Escenario:

1. Usuario visita sitio del adversario (bad.com)
2. Adversario visita sitio real (bank.com), obtiene desafío
3. bad.com envía desafío al usuario
4. Dispositivo del usuario firma desafío
5. bad.com envía desafío firmado a bank.com
6. Login exitoso

Solución: Vincular desafío a identidad del servidor

Protocolo mejorado:

```
S -> B: desafío
B -> D: CD = {desafío, origen}
    donde origen = H(protocolo || hostname || puerto)
D -> B: Sign(Kpriv, CD) -> firma s
B -> S: s, CD
S: Verify(Kpub, CD, s), verificar CD contiene desafío y origen esperados
```

Adversario causará que navegador envíe CD diferente, así que firma diferente.

3.18 Problema 2: Vinculación entre Sitios

Preocupación de privacidad:

- Servidores pueden comparar valores Kpub
- Aprender qué usuarios son los mismos entre sitios
- No se podría decir que usuario es el mismo con contraseñas

Solución: Registro de claves por sitio

- Al registrarse, dispositivo genera clave fresca (Kpub, Kpriv)
- Dispositivo envía Kpub en claro
- Dispositivo envía Kpriv encriptado con Kwrap (clave maestra)
- Kwrap almacenado en dispositivo (única clave de largo plazo)
- Encriptación de Kpriv está asociada con origen
- "Handle"^{es} este Kpriv encriptado
- En desafío posterior:
 - Dispositivo desencripta handle en Kpriv
 - Verifica origen

3.19 Otros Ataques

Atacante compromete el cliente:

- Podría robar cookie de sesión después del login
- Podría engañar al usuario para presionar botón de clave de seguridad

Atacante roba el dispositivo:

- Aún necesita contraseña del usuario (si es fuerte)

Atacante suministra dispositivo malo:

- Atestación del dispositivo detectaría esto y evitaría registro

BONUS**WebAuthn y FIDO2:**

WebAuthn es la evolución moderna de U2F:

Mejoras sobre U2F:

- Soporte para biométricos (Touch ID, Face ID, Windows Hello)
- Autenticadores de plataforma (integrados en dispositivo)
- Autenticadores externos (USB, NFC, Bluetooth)
- Passwordless authentication posible

Adopción:

- Soporte en todos los navegadores modernos
- Implementado por grandes servicios (Google, Microsoft, GitHub, etc.)
- Estándar W3C desde 2019

Ventaja clave: Resistente a phishing por diseño, a diferencia de contraseñas y TOTP.

4 Arquitectura de Seguridad: Google

4.1 ¿Qué es Arquitectura de Seguridad?

Estructuración de sistemas completos para:

- Defender contra grandes clases de ataques conocidos
- Prevenir ataques aún desconocidos
- Contener daños de ataques exitosos
- Adelantarse a los atacantes, no solo reaccionar

4.2 La Arquitectura de Seguridad Consiste en:

Formas de analizar la situación:

- ¿Qué estamos defendiendo? (tarjetas de crédito, claves cripto, secretos comerciales, todo)
- ¿Quién es el atacante? (spammers, empleados, vendedores, clientes, competidores)
- ¿Qué poderes asumimos que el ataque tiene/no tiene?
- Todo esto = **Modelo de Amenaza**

Principios:

- Minimizar confianza

Técnicas:

- Aislamiento, autenticación, separación de privilegios
- Canales seguros

4.3 Caso de Estudio: Paper de Google

Enfoque: Plataforma en la nube de Google

Objetivos de seguridad:

- Evitar divulgación de datos del cliente
- Asegurar disponibilidad de aplicaciones
- Rastrear qué salió mal si ocurre compromiso
- Ayudar a ingenieros a construir aplicaciones seguras
- Asegurar que los clientes confíen en Google

Amenazas:

- Errores en software de Google
- Redes comprometidas (cliente, Internet, interna)
- Contraseñas de empleados robadas
- Malware en estaciones de trabajo/smartphones
- Ataques internos (sobornar ingeniero/operador)
- Hardware de servidor malicioso
- Datos en discos descartados

4.4 Entorno de Servidor de Google

- Centros de datos
- Máquinas físicas
- Máquinas virtuales
- Servicios en VMs
- Aplicaciones (Google y cliente) en VMs
- RPC entre aplicaciones y servicios
- Servidores front-end convierten HTTP/HTTPS en RPC

4.5 Aislamiento: Punto de Partida para Seguridad

Objetivo: Por defecto, actividad X no puede afectar actividad Y

- Incluso si X es maliciosa
- Incluso si Y tiene errores

Sin aislamiento: No hay esperanza para seguridad

Con aislamiento: Podemos permitir interacción controlada

Ejemplos en diseño de Google:

- Separación de usuarios de Linux

- Sandboxes de lenguaje
- Sandboxes de kernel
- Máquinas virtuales (VMs)
- Máquinas dedicadas para servicios sensibles
- Service Identity, Integrity, and Isolation

4.6 Máquinas Virtuales para Aislamiento

Arquitectura:

- Cada máquina física tiene VMM (Virtual Machine Monitor) host
- VMM supervisa muchas VMs guest
- Cada VM guest ejecuta un OS, etc.

Ventajas:

- Permite compartir máquinas entre actividades no relacionadas
- Muchas actividades solo necesitan fracción de máquina

Punto 1: Mantener fuera

- VMM ayuda a mantener fuera atacantes en otras VMs guest
- Ej: Servidor de almacenamiento en una VM, cliente en otra VM
- Ej: Almacenamiento en VM, servicio comprometido en otra VM

Punto 2: Confinamiento

- VMM ayuda a mantener atacantes *dentro*
- Seguro ejecutar casi cualquier código en VM guest, incluso en kernel
- Mientras se controle con quién habla por red

¿VMs proporcionan aislamiento perfecto?

- No - existen ataques de escape de VM
- Pero eleva significativamente la barrera

4.7 Compartir: Monitor de Referencia

100 % de aislamiento no es lo deseado. Necesitamos compartir/interacción controlada.

Modelo del Monitor de Referencia:

Principal -> [Solicitud] -> [GUARDIA] -> Recurso

Componentes:

- **Principal:** Persona, dispositivo, programa, servicio
- **Recurso:** Servicios (ej. servicio de Contactos), elementos dentro de servicios (archivos)
- **Guardia:** Auténtica, autoriza, audita

4.8 Autenticar: ¿Quién/Qué emite la solicitud?

¿Cómo autenticar a una persona?

1. Contraseñas

- ¿Qué aprende el Guardia de contraseña correcta?
- ¿Podemos hacerlo mejor?

2. Autenticación de dos factores

- Ej: enviar número al usuario vía SMS
- ¿Qué aprende el Guardia de 2FA correcta?
- ¿Podemos hacerlo mejor?

3. Llaves públicas y privadas

- Más resistente a phishing

4.9 Autorizar: Determinar si Solicitud Debe Ser Permitida

Función de política: permisos = POLÍTICA(principal, recurso)

Equivalente: Matriz de acceso

	Alice	Bob	...
File1	Y	N	
File2	Y	Y	
...			

Dos formas de almacenar política:

1. Access Control Lists (ACLs): Almacenar fila (cortar por recurso)

- Para política de largo plazo
- Permisos de archivo, lista de usuarios con acceso a documento
- Almacenados con objeto protegido
- Buenos para "¿quién puede acceder a este recurso?"

2. Capacidades: Almacenar columna (cortar por principal)

- Básicamente qué puede hacer un principal
- Short term
- Para política de corto plazo
- Descriptores de archivo en OS, referencias de objeto en lenguajes
- Tokens criptográficos en protocolos distribuidos
- Almacenados con principal
- Flexibles - aplicaciones pueden otorgarse capacidades entre sí
- No buenos para largo plazo: no responden "¿quién tiene acceso?", revocación complicada

4.10 Granularidad de Autorización

1. Nivel de Data Center

- Confiamos en lo que está adentro → Firewall

2. Nivel de Servicio

- Nivel mínimo de privilegios

3. Nivel de Usuario

- Autorización (lo que ya vimos)

4.11 Ejemplos de Autorización en Google

Inter-Service Access Management:

- ACL: Administrador white-lista quién puede usar cada servicio
- Principales = otros servicios, ingenieros
- Guardia = aplicación automática por infraestructura RPC

Ticket de permiso de usuario final:

- Capacidad para realizar operaciones en nombre de usuario final
- Ej: para acceder al servicio de Contactos
- Ticket de corta duración para limitar daños si es robado

4.12 ¿Por qué la estructura del Monitor de Referencia?

- Separa política de implementación del recurso
 - Facilita razonamiento
 - Facilita evolución de política
- Implicación: Evitar incrustar verificaciones de seguridad en código del recurso
 - Aunque incrustar sea conveniente!
- Nota: Depende del aislamiento - no hay acceso excepto a través del guardia

4.13 Defensa Perimetral vs Monitor de Referencia

Defensa perimetral antigua:

- Aislar "dentro de empresa" de "fuera de empresa"
- Solo una conexión (lógica) a Internet
- Firewall actúa como guardia
- Acceso completo permitido dentro del firewall
- Funcionó bien durante muchos años

Debilidades:

- No hay historia para nada que salga mal dentro
- No hay segunda línea de defensa si hay penetración exitosa

Enfoque de Google:

- Aplicar Monitor de Referencia por servicio
- Guardia = aceptar RPCs solo de servicios cliente aprobados
- Ej: GMail puede hablar con Contactos, otros servicios no
- Ejemplo de Privilegio Mínimo

4.14 Tickets de Permiso de Usuario Final

Reducen privilegio aún más:

- RPC debe ser de servicio aprobado + usuario realmente logueado
- Motivación: Limitar daños de servicios con errores
- Motivación: Limitar capacidad de internos de acceder datos arbitrarios
- Quizás tickets también están atados al cifrado de datos

4.15 ¿Cómo Autentica un Servicio a Otro?

Necesitamos canales seguros”sobre la red.

Criptografía: cifrar/descifrar, firmar/verificar

- Firmas prueban quién envió mensaje (integridad)
- Cifrado asegura que solo destinatario pueda leer (confidencialidad)

En Google:

- Servidores (probablemente) firman RPCs a otros servidores
- Sistema RPC automáticamente limita quién puede hablar con quién
- RPCs cifrados entre centros de datos, sobre internet
- Quizás también cifrados dentro del centro de datos (¿por qué?)

4.16 Criptografía y Gestión de Claves

El desafío:

- Remitente RPC necesita saber qué clave usar para cifrar
- Receptor RPC necesita saber quién corresponde a clave de firma
- Google ejecuta servicio de nombres, mapeando nombres de servicio a claves públicas

4.17 Arranque Seguro de Servidores

¿Cómo sabe Google que es seguro usar computadora particular?

Usar servidor significa confiar en hardware/BIOS/etc:

- Datos sensibles, claves cripto, autorización RPC

¿Por qué podría haber problema?

- Atacante intercambia físicamente su servidor por uno de Google
- Atacante irrumpe, cambia el OS en disco
- Vendedor envía máquina con BIOS corrupto
- Atacante irrumpe, "actualiza" BIOS a algo malo

Estrategia de defensa de Google:

- Diseñan sus propias placas base
- Diseñan su propio chip de seguridad
- Chip interviene durante proceso de arranque
- Chip verifica que BIOS y OS estén firmados por clave privada de Google
- Chip tiene clave privada única atada a máquina particular
- Chip dispuesto a firmar declaraciones cuando se le pide
- Declaración incluye identidad (hash) del BIOS y OS arrancados
- Servicios requieren autenticación incluyendo declaración firmada
- Servicios pueden verificar declaración:
 - Google tiene BD de claves públicas del chip para todas las máquinas compradas
 - Google tiene BD de hashes aceptables de BIOS y OS

¿Qué pasa si empleado inserta máquina con dirección IP correcta?

- Sin clave pública registrada, será rechazada

¿Qué pasa si vendedor envía máquina con BIOS que espía?

- Hash de BIOS no coincidirá con BD de hashes aprobados

4.18 Disponibilidad: Ataques DoS

El problema:

- Atacante quiere sacar sitio web del aire o chantajear
- Ensamblan "botnet" de 10,000 máquinas
- Envían vastas cantidades de solicitudes

Recursos objetivo de ataque DoS:

- Ancho de banda de red

- CPU/memoria del router (paquetes pequeños, opciones inusuales, protocolos de enrutamiento)
- Memoria del servidor (estado del protocolo - SYN floods)
- CPU del servidor (acciones de aplicación costosas)

Problema central: Difícil distinguir tráfico de ataque del tráfico real

Principios amplios para mitigar:

- Recursos masivos del lado del servidor, con distribución/balance de carga
- Autenticar lo antes posible
- Minimizar consumo de recursos antes de autenticación
- Factorizar componentes que manejan solicitudes antes de autenticación (Google: GFE, servicio de login)
- Limitar/priorizar uso de recursos después de autenticación
- Usuarios legítimos autenticados deben obtener prioridad
- Google implementa varias heurísticas para filtrar solicitudes en GFE

BONUS

Arquitectura de Zero Trust:

Google fue pionero en el modelo BeyondCorp (Zero Trust):

Principios:

- Acceso basado en identidad del dispositivo y usuario, no ubicación de red
- Todos los dispositivos y usuarios son no confiables por defecto
- Cada solicitud es autenticada, autorizada y encriptada
- Acceso es otorgado basándose en políticas dinámicas

Componentes:

- Inventario de dispositivos y verificación de salud
- Autenticación fuerte de usuario y dispositivo
- Políticas de acceso granulares basadas en contexto
- Microsegmentación de recursos
- Monitoreo y análisis continuo

Ventajas:

- Elimina la necesidad de VPN tradicional
- Reduce superficie de ataque
- Mejora experiencia de usuario
- Facilita trabajo remoto seguro

Implementación requiere:

- Cambio cultural significativo
- Inventario completo de activos
- Sistema robusto de gestión de identidad
- Proxy de acceso (como Google Identity-Aware Proxy)

5 OKWS: Separación de Privilegios

5.1 El Problema: ¿Qué Hacer con los Bugs?

Plan A: Encontrarlos, corregirlos, evitar crear nuevos

- Gran progreso con buffer overflows
- ¿Es este plan suficiente? No

Plan B: Construir sistemas seguros incluso si hay bugs

- Gran idea: separación de privilegios
- Dividir software y datos para limitar daño de bugs

Dos beneficios relacionados:

1. Limitar daño de exploit exitoso - "menor privilegio"
2. Limitar acceso del atacante al código con bugs - "superficie de ataque"

5.2 Configuración Tradicional: Apache

Arquitectura típica:

- Apache ejecuta N procesos idénticos
- Cada proceso tiene todo el código de aplicación:
 - Ejecuta solicitudes para muchos usuarios
 - Ejecuta muchos tipos diferentes de solicitudes (login, leer correo, etc.)
- Almacenamiento: Base de datos SQL con contraseñas, cookies, mensajes

Esta disposición es:

- Conveniente y eficiente
- Segura solo si código web no tiene bugs

¿Qué puede salir mal?

- Buffer overflow + inyección de código expone toda la DB
- Bugs de memoria permiten leer datos de memoria de app
- Verificaciones de control de acceso faltantes antes de consultas DB
- Bugs en manejo de archivos dan acceso a archivos sensibles
 - Ej: `open(/profiles/Γ user)`
 - ¿Qué pasa con `user="../../../etc/passwd` o `"../mail/rtm"`?
- Inyección SQL permite leer/escribir todos los datos DB
 - `"SELECT email FROM users WHERE id = Γ userid`

Configuración "cáscara dura, interior blando" hace que bugs sean devastadores

5.3 Separación de Privilegios

El diseñador debe elegir esquema de separación:

- Por servicio / tipo de datos (listas de amigos vs contraseñas)
- Por usuario (mi correo vs tu correo)
- Por cantidad de bugs (redimensionamiento de imágenes vs todo lo demás)
- Por exposición a ataque directo (parsing HTTP vs todo lo demás)
- Por privilegio inherente (ocultar procesos superusuario, ocultar DB)

La separación de privilegios es difícil:

- Necesita aislar (cliente/servidor, VMs, contenedores, procesos)
- Necesita permitir interacción controlada
- Necesita retener buen rendimiento

5.4 Principios de OKWS

1. Los procesos deben correr *chrooted*
2. Los procesos deben correr como usuarios no privilegiados
3. Los procesos deben tener privilegios mínimos de BD
4. Funcionalidad independiente debe estar en procesos independientes

5.5 ¿Cómo Particiona OKWS el Servidor Web?

Componentes principales:

- **OK Launcher Daemon (okld):** Servicio a cargo de lanzar otros servicios
- **OK Dispatcher (okd):** Rutea requests al servicio correspondiente
- **pubd:** Acceso a archivos (templates, etc.)
- **OK Logger Daemon (oklogd):** Escribe logs
- **dbproxy:** Intermediario a la BD
- **Servicios:** matching, messaging, editor de perfil, fotos, login

Características de seguridad:

- Todos los servicios con *chroot*
- Cada servicio corre bajo usuario sin privilegios
- Todos los requests a BD a través de RPC

Arquitectura física:

- Muchas máquinas front-end: okld, okd, pubd, oklogd, servicio*
- Pocas máquinas DB: dbproxy, DB

5.6 ¿Qué Daño si Cada Componente es Comprometido?

okld:

- Privilegios: Acceso superusuario a máquina del servidor web
- Superficie de ataque: Pequeña (sin entrada de usuario excepto salida del servicio)

okd:

- Privilegios: Interceptar todas solicitudes/respuestas HTTP, robar contraseñas
- Superficie de ataque: Parsear primera línea de solicitud HTTP

pubd:

- Privilegios: Algo de acceso al sistema de archivos, podría corromper plantillas
- Superficie de ataque: Solicitudes para obtener plantillas de okd

oklogd:

- Privilegios: Cambiar/eliminar entradas de log - cubrir huellas
- Superficie de ataque: Mensajes de log de okd, okld, servicios

service:

- Privilegios: Datos del servicio para cualquier usuario, solicitudes a dbproxy
- Superficie de ataque: Solicitudes HTTP, contenido DB (!)

dbproxy:

- Privilegios: Acceder/cambiar todos datos en BD con la que habla
- Superficie de ataque: Solicitudes de servicios autorizados, solicitudes de servicios no autorizados (fácil de descartar)

5.7 ¿Dónde Debería Buscar un Atacante Debilidades?

Probablemente muchos bugs en implementaciones de servicio:

- Tal vez no tan malo para servicio de "tipo de relación"
- Malo si bug en servicio de correo (puedo leer/escribir tu correo)
- Muy malo si bug en servicio de autenticación de contraseñas
- Ojalá servicios sensibles tengan pocas líneas de código (pocos bugs)

Otros vectores de ataque:

- Bugs en kernel del OS
 - Código inyectado podría explotar bug del kernel
 - Para convertirse en superusuario, salir del chroot
- Bugs en parsing de URL de okd
- Bugs en proxies DB (inyección SQL, demasiado permisivo)

5.8 ¿Por Qué Proxies de Base de Datos Separados?

¿Por qué no dejar que servicios hablen con DB directamente?

- DB acepta consultas SQL generales, puede recuperar/modificar cualquier cosa
- dbproxy acepta RPCs (no SQL); dbproxy genera SQL y habla con DB
 - Así código del servicio no está sujeto a inyección SQL
- dbproxy sabe qué consultas se permite hacer a cada servicio
 - Aquí vive la política de seguridad: matriz servicio/consulta
- Un desarrollador conocedor debe mantener dbproxy

5.9 ¿Cómo Sabe dbproxy Qué Servicio Está Hablando?

Problema:

- dbproxies en máquinas DB separadas
- Servicios usan sockets TCP para conectar
- No puedes decir mucho desde TCP sobre quién se conectó

Solución:

- Cada servicio tiene token secreto único de 20 bytes
- Servicio suministra su token en RPC cuando habla con dbproxy
- dbproxy tiene lista de consultas permitidas para cada token
- ¿De dónde viene el token? okld lee desde config, lo pasa al servicio

5.10 Falta de Aislamiento por Usuario en OKWS

El paper argumenta que aislamiento por usuario es demasiado costoso:

- Costo de iniciar proceso (o mantener proceso ejecutándose) es alto
- Comparado con costo de procesar solicitud HTTP

Truco/tradeoff potencial del paper de Google:

- Un servicio ejecutándose pero solo puede actuar en nombre de algunos usuarios
- "Ticket de permiso de usuario final"

¿Tendrían sentido permisos de usuario estilo Google en OKWS?

Desde perspectiva de seguridad:

- Servicio no puede acceder datos de usuarios que no han accedido a ese servicio
- Servicio con bugs raramente usado no puede causar tanto daño
- Servicio comprometido tiene que esperar a que usuarios inicien sesión
- Toma más tiempo para adversario obtener datos de muchos usuarios
- Más oportunidad para operadores noten ataque, se recuperen

5.11 Mecanismos de Aislamiento en UNIX

¿Cuáles son los mecanismos?

- Procesos Unix, IDs de usuario (UIDs), permisos de archivo, paso de fd

¿Qué es `setuid(uid)`?

- Un proceso puede soltar sus privilegios de root a uid ordinario

¿Qué es `chroot(dirname)`?

- Hace que `/` se refiera a `dirname` para este proceso y descendientes
- Así no pueden nombrar archivos fuera de `dirname`

¿Qué es paso de FD?

- Un proceso abre conexión de red
- Pasa descriptor de archivo a otro proceso
- Ej: `okld` pasa descriptor de archivo para puerto 80 a `okd`

5.12 ¿Cómo Hace Cumplir OKWS el Aislamiento?

Tabla 1: Configuración de `chroot`

Componente	<code>chroot</code>	UID/GID
<code>okld</code>	<code>/okws</code>	root
<code>okd</code>	<code>/okws</code>	<code>okd</code>
servicios	<code>/okws</code>	servicio-específico
<code>pubd</code>	<code>/pub</code>	<code>pubd</code>
<code>oklogd</code>	<code>/logs</code>	<code>oklogd</code>

Cómo se hace cumplir:

- `okld` ejecuta cada servicio con UID separado
- En lab 2, ejecutarías cada servicio como contenedor separado
- Así servicios no pueden leer/escribir memoria de cada uno
- `okld` usa `chroot` para prevenir que procesos vean mayoría de archivos
- `okld` ejecuta como root (para `setuid()` y para asignar puerto TCP 80)
- ¡Así que queremos que haga lo menos posible!

¿Por qué `okd` es proceso separado?

- Necesitamos forma de enrutar solicitudes HTTP al servicio correcto
- `okd` ve todas solicitudes, así que no queremos hacer nada más en `okd`
- Nota: `okd` NO ejecuta como superusuario; `okld` le da puerto 80

¿Por qué `oklogd` es proceso separado?

- No queremos que servicio corrupto elimine/sobrescriba archivos de log

¿Por qué `pubd` es proceso separado?

- Mantiene código de manejo de archivos fuera de servicios

5.13 ¿Por Qué okld, okd y Servicios Comparten chroot?

- Queremos hacer chroot a okld (puede tener bugs también)
- okld necesita relanzar okd + servicios
- Así okd y servicios necesitan vivir en cárcel chroot de okld

¿Qué estamos exponiendo?

- Legible: Bibliotecas compartidas conteniendo código de servicio
- Escribible: Cada servicio puede escribir a su propio `/cores/<uid>`

¿Dónde está el archivo config?

- `/etc/okws_config`, okld lee al inicio antes de chroot

oklogd & pubd tienen chroots separados porque usan archivos:

- okld debe iniciar oklogd y pubd antes de hacer chroot a sí mismo

¿Por qué UID separado para cada servicio?

- kill, ptrace, archivos core

¿Por qué GID separado para cada servicio?

- Así servicio puede ejecutar su binario pero no leer/escribir/chmod
- Binario propiedad de root, y x-solo para GID del servicio: `rwX--X---`
- Así servicio no puede leer secretos de su ejecutable
- No puede modificarlo para persistir ataque

5.14 ¿Cómo Interactúan los Componentes OKWS?

- okld configura socketpairs (pipes bidireccionales) para cada servicio
 - RPCs de control
 - Logging
 - Conexiones HTTP, okd → servicio
- Los servicios hablan con proxy DB sobre TCP
- Mayoría del estado en DB, mayoría de interacción vía estado en DB

5.15 Dificultades con Herramientas de Aislamiento UNIX

- Muchos espacios de nombres globales: archivos, UIDs, PIDs, puertos
 - Cada uno puede permitir que procesos vean lo que otros están haciendo
 - Cada uno es invitación para bugs o configuración descuidada
- No hay idea de "por defecto sin acceso"
 - Difícil para diseñador razonar sobre qué puede hacer un proceso
- No hay concesiones granulares de privilegio
 - No puedes decir "proceso puede leer solo estos tres archivos"
- `chroot()` y `setuid()` solo pueden ser usados por superusuario
 - No-superusuarios no pueden reducir/limitar su propio privilegio
 - Incómodo ya que seguridad sugiere NO ejecutar como superusuario

5.16 ¿Qué Ha Pasado Desde 2004?

OKWS mismo:

- Aún (probablemente) usado en OK Cupid
- No usado en otros lugares
- C++ no es popular para programación web
- Herramientas de aislamiento UNIX difíciles de usar
- Partición fina es difícil, tensión con desarrollo rápido
- Partición OKWS no muy útil si solo tienes un servicio importante
- Liberación open-source deprecada por mantenedores

Separación de privilegios comúnmente usada en práctica:

- Balanceador de carga, servicio de login, servicio de perfil
- DB de contraseñas, DB de perfil
- Paper de arquitectura de Google es buen ejemplo
- Incluso más granulado que OKWS: tickets por usuario

Sistemas con partición grano fino estilo OKWS:

- proceso ssh-agent para mantener claves crypto, vs ssh mismo
- Chrome ejecuta cada frame en proceso aislado
- VMs, FreeBSD jail, contenedores Linux (Docker)
- Linux seccomp, etc.

BONUS

Lab 2 usa Contenedores Linux (lxc):

No existían cuando autor construyó OKWS

Contenedores proporcionan ilusión de máquinas virtuales sin usar VMs:

- Más eficientes que máquinas virtuales
- Un contenedor es proceso Linux, pero fuertemente aislado
- Acceso limitado a espacios de nombres del kernel
- Acceso limitado a llamadas al sistema
- Sin acceso al sistema de archivos

Contenedores se comportan como VM:

- Iniciados desde imagen VM
- Tienen su propia dirección IP
- Tienen su propio sistema de archivos
- Lab 2 usa contenedores *no privilegiados*
- Más difícil salir del contenedor que proceso con chroot

Usar contenedores para separación de privilegios:

- Plan: Convertir aplicación de proceso único en aplicación virtual "distribuida"
- Crear contenedor para diferentes servicios
- Limitar comunicación entre contenedores (reglas de firewall)
- Lab 2 tiene zookld que hace esto; similar a okld

6 Linux Containers

6.1 Recordatorio: Separación por Procesos

- Vimos separación por procesos (OKWS)
 - Difícil de manejar
- Separación de Privilegios
 - Mínimo Privilegio
 - Separación de responsabilidades

6.2 chroot

- Cambia el directorio raíz
 - Por ejemplo de / a /home/ignacio
- No es suficiente
 - Es aislación que se puede romper

6.3 Casos de Uso de Contenedores

- Segregación de privilegios más fuerte en arquitectura de microservicios en un solo host
- Mejor contención del radio de explosión en caso de compromiso de seguridad
- Utilización de recursos más efectiva en aislamiento (comparado con virtualización asistida por hardware)
- Facilidad de despliegue de software (propósito original de contenedores)

6.4 Máquinas Virtuales vs Containers

Diferencia fundamental:

- **VMs:** Una CPU con muchos kernels
- **Containers:** Un kernel con muchos espacios de usuario

¿Qué es el kernel?

- Núcleo del sistema operativo
- Gestiona recursos de hardware
- Proporciona servicios a aplicaciones

6.5 Linux Containers

chroot en esteroides:

- Grupo de procesos
- Usa **namespaces** para vistas privadas
- Usa **cgroups** para limitar uso de recursos

- Usa **capabilities** para limitar lo que puede hacer root
- Usa **seccomp-bpf** para filtrar syscalls del kernel

Características:

- Ligeros, rápidos y dispensables
- Corren en milisegundos
- Usan Mb (no Gb como VMs)

6.6 Namespaces

Concepto: Un namespace hace que un recurso solo pueda tener acceso a otros recursos en el mismo namespace

Ejemplo: Separar procesos dentro de namespaces separados para que no se puedan ver entre sí

Tipos de namespaces:

- **PID:** Separa procesos
- **UTS:** Aisla hostnames (fqdn)
- **cgroup:** Aisla cgroups
- **IPC:** Aisla comunicación entre procesos
- **User:** Aisla usuarios
- **Mount:** Aisla mountpoints
- **Network:** Aisla la red

6.7 Control Groups (cgroups)

Propósito: Limita la cantidad de recursos que un grupo de procesos puede utilizar

Utilidad: Por ejemplo útil para multi-tasking

Capacidades:

1. **Límites:** Se pueden especificar límites máximos
 - Uso del procesador
 - Uso de memoria
 - Uso de dispositivos
2. **Contabilidad:** Se monitorea uso de recursos
3. **Priorización:** Uso de recursos puede ser priorizado sobre otros cgroups
4. **Control:** Estado de procesos puede ser controlado
 - Detener, reiniciar, suspender

Se puede limitar:

- Memoria
- CPU

- Disco
- Dispositivos
- Acceso a la red
- etc.

6.8 Capabilities

Antes:

- Usuario *root* (privilegiado) vs usuario no privilegiado

Problema:

- ¿Qué pasa si necesitamos que usuario no privilegiado corra algo con permisos privilegiados?

Solución: Capabilities

Ejemplos de capabilities:

- **CAP_CHOWN:** Cambiar propietario de archivos
- **CAP_NET_ADMIN:** Operaciones de red administrativas
- **CAP_NET_RAW:** Usar sockets RAW (ej. ping)

6.8.1 Ejemplo: ping

```
1 ignacio@ubuntu:~$ ls -al /bin/ping
2 -rwxr-xr-x 1 root root 72776 Jan 30 15:11 /bin/ping
3
4 USER      PID  %CPU %MEM    VSZ   TTY      COMMAND
5 ignacio  3220  0.0   0.0   18464 pts/0    /bin/ping google.com
6
7 ignacio@ubuntu:~$ getcap /bin/ping
8 /bin/ping = cap_net_raw+ep
```

6.8.2 Tipos de Capabilities

- **Effective (e):** Capability activa
- **Inheritable (i):** Es heredada por los hijos
- **Permitted (p):** Permitida sin importar capability del padre

6.8.3 Ejemplo: Asignar Capability

```
1 ignacio@ubuntu:~$ cp /bin/ping .
2 ignacio@ubuntu:~$ getcap ./ping
3 ignacio@ubuntu:~$ ./ping google.com
4 ping: socket: Operation not permitted
5
6 ignacio@ubuntu:~$ sudo setcap cap_net_raw=ep ./ping
7 ignacio@ubuntu:~$ ./ping google.com
8 PING google.com (172.217.11.14) 56(84) bytes of data.
```

6.9 seccomp-bpf

Origen: Originalmente para networking (Berkeley Packet Filter)

Propósito: Limita los syscalls que un proceso puede hacer

Escenario:

- Atacante toma control del contenedor
- Logra tener privilegios de root
- Queremos evitar que haga syscalls peligrosas

6.10 ¿Cómo Funciona Todo Junto?

- **Namespaces:** Aislan elementos del kernel
- **Capabilities:** Reducen privilegios no deseados
- **cgroups:** Limitan recursos de hardware
- **seccomp-bpf:** Filtran llamadas al kernel

Resultado: Contenedor con privilegios limitados

BONUS

Docker vs Kubernetes:

Docker:

- Plataforma para construir, distribuir y ejecutar contenedores
- Usa Linux containers bajo el capó
- Simplifica creación de imágenes (Dockerfile)
- Registry para compartir imágenes (Docker Hub)
- Bueno para desarrollo local y aplicaciones simples

Kubernetes (K8s):

- Sistema de orquestación de contenedores
- Gestiona múltiples contenedores a través de múltiples hosts
- Características: auto-scaling, self-healing, rolling updates
- Necesario para aplicaciones de producción a escala
- Puede usar Docker u otros runtimes de contenedores

Seguridad en Kubernetes:

- Pod Security Policies/Standards
- Network Policies para microsegmentación
- RBAC (Role-Based Access Control)
- Secrets management
- Image scanning y admission controllers

Herramientas de seguridad:

- Falco: Detección de amenazas en runtime
- Trivy: Escaneo de vulnerabilidades
- Open Policy Agent: Políticas como código
- Istio: Service mesh con mTLS automático

Parte II

SDLC Seguro: De Código a Release

7 Hacia un SDLC Seguro

El Software Development Lifecycle (SDLC) tradicional se enfoca en entrega rápida de funcionalidades, pero seguridad debe integrarse en cada etapa.

Un SDLC seguro permite:

- Reducir costos de corrección de vulnerabilidades
- Minimizar riesgo de brechas de seguridad
- Cumplir con regulaciones y estándares
- Generar confianza con usuarios y stakeholders

BONUS

Costo de corregir vulnerabilidad aumenta exponencialmente:

- **Diseño:** 1x (costo base)
- **Implementación:** 6x
- **Testing:** 15x
- **Producción:** 30-100x

Esto justifica inversión temprana en seguridad.

8 Vulnerabilidades Comunes en el SDLC

8.1 Herramientas DevOps e Infraestructura

Configuraciones por defecto priorizan eficiencia y velocidad, no seguridad:

- Contenedores ejecutándose como root
- Pipelines de CI/CD sin validación de seguridad
- Secretos hardcodeados en código fuente
- Políticas de acceso permisivas

8.2 Manipulación de Código

Alteración no autorizada del código e inyección de código malicioso:

- Compromiso de cuentas de desarrolladores
- Ataques a cadena de suministro
- Repositorios sin protección de ramas
- Falta de revisión de código

BONUS

Caso real: SolarWinds (2020)

Atacantes comprometieron proceso de build de SolarWinds, inyectando código malicioso en actualizaciones legítimas del software Orion. Afectó a más de 18,000 organizaciones.

Lecciones aprendidas:

- Firmado de código con múltiples llaves
- Verificación de integridad en cada etapa
- Aislamiento de ambientes de build
- Auditoría continua de procesos de compilación

8.3 Prácticas Inseguras de Codificación

- Exposición de credenciales en código o logs
- Falta de validación de entradas (SQL injection, XSS)
- Control de acceso inadecuado
- Configuraciones de seguridad incorrectas
- Uso de componentes con vulnerabilidades conocidas

8.4 Filtraciones de Código y Datos Sensibles

- Publicación accidental o maliciosa de código privado
- Credenciales en commits
- Divulgación de datos de usuarios en logs/debugging
- Propiedad intelectual

BONUS

Prevención de filtraciones:

- Usar herramientas como git-secrets, truffleHog
- Implementar pre-commit hooks
- Escaneo continuo de repositorios públicos
- Políticas de clasificación de información
- Rotación automática de secretos detectados

9 Gobernanza de Seguridad en el SDLC

9.1 Visibilidad

Seguridad debe conocer y tener acceso a:

- Todas las herramientas DevOps utilizadas
- Personal involucrado en desarrollo y operaciones
- Procesos y flujos de trabajo del SDLC
- Inventario de aplicaciones y servicios

9.2 Políticas de Privilegios Mínimos

Desarrollo y Operaciones con acceso mínimo necesario:

- Permisos basados en roles (RBAC)
- Acceso just-in-time cuando sea posible
- Revisión periódica de permisos
- Revocación automática de accesos inactivos

BONUS**Implementación de Zero Trust:****Principios:**

1. Verificar explícitamente (nunca confiar automáticamente)
2. Usar acceso de menor privilegio
3. Asumir que ha ocurrido una brecha

En contexto SDLC:

- Autenticación continua
- Microsegmentación de ambientes
- Monitoreo de comportamiento anómalo
- Validación en cada transacción

9.3 Autenticación

- Multi-Factor Authentication (MFA) obligatorio
- Single Sign-On (SSO) para gestión centralizada
- Llaves de seguridad física (FIDO2/U2F)
- Políticas de contraseñas fuertes

9.4 Protección de Ramas y Reglas de Compilación

Controles en repositorios:

- Revisión obligatoria de código (peer review)
- Firmado de commits con GPG
- Prohibición de pushes forzados a ramas principales
- Prohibición de repositorios personales para código corporativo
- Búsqueda automatizada de credenciales y secretos
- Ejecución obligatoria de pruebas unitarias
- Validación de cobertura de código mínima

9.5 Supervisión de Cambios

- Solicitudes de cambios requieren aprobación de Seguridad
- Alertas ante cambios no autorizados
- Auditoría completa de modificaciones
- Reversión automática de cambios no conformes

10 Shift Left Security y DevSecOps

10.1 Conceptos Fundamentales

Ambos buscan integrar seguridad en cada etapa del SDLC, con alcances complementarios.

10.1.1 Shift Left Security

Detectar y corregir problemas de seguridad en etapas tempranas:

- Reduce costo y esfuerzo de corrección
- Previene que vulnerabilidades lleguen a producción
- Enfoca en herramientas automatizadas tempranas

10.1.2 DevSecOps

Hacer de seguridad una responsabilidad compartida durante todo el SDLC:

- Involucra desarrolladores, operaciones y seguridad
- Cultura de colaboración
- Automatización de controles de seguridad
- Retroalimentación continua

BONUS

Los tres pilares de DevSecOps:

1. Personas:

- Capacitación continua en seguridad
- Champions de seguridad en cada equipo
- Cultura de aprendizaje sin culpa

2. Procesos:

- Integración de seguridad en workflows existentes
- Automatización de tareas repetitivas
- Feedback loops rápidos

3. Tecnología:

- Herramientas integradas en pipeline
- Observabilidad y métricas de seguridad
- Infrastructure as Code (IaC) seguro

11 Modelado de Amenazas

Proceso estructurado para identificar y mitigar riesgos. Cuatro preguntas fundamentales:

11.1 ¿Qué estamos construyendo?

- Diagramas de arquitectura
- Flujos de datos
- Componentes y sus interacciones
- Activos críticos

11.2 ¿Qué puede salir mal?

Metodología STRIDE para identificar violaciones:

- **Spoofing:** Suplantación de identidad
- **Tampering:** Manipulación de datos
- **Repudiation:** No reconocimiento de acciones
- **Information Disclosure:** Divulgación de información
- **Denial of Service:** Denegación de servicio
- **Elevation of Privilege:** Escalada de privilegios

11.3 ¿Qué haremos al respecto?

Definir controles y mitigaciones:

- Controles preventivos
- Controles detectivos
- Controles correctivos
- Aceptación de riesgos documentada

11.4 ¿Lo hicimos bien?

Validación y revisión:

- Revisión por pares
- Testing de controles implementados
- Actualización continua del modelo
- Documentación de decisiones

BONUS

Herramientas para Modelado de Amenazas:

- **Microsoft Threat Modeling Tool:** Gratuita, integrada con STRIDE
- **OWASP Threat Dragon:** Open source, basada en web
- **IriusRisk:** Comercial, automatización avanzada
- **Threagile:** YAML-based, as-code approach

Ejemplo de amenaza STRIDE:

Spoofing en login:

- Amenaza: Atacante falsifica identidad de usuario legítimo
- Impacto: Acceso no autorizado a datos sensibles
- Mitigación: MFA, monitoreo de intentos fallidos
- Validación: Penetration testing, revisión de logs

12 SLS: Planificación, Análisis y Diseño

12.1 Planificación

Involucrar equipo de seguridad desde el inicio:

- Participación en reuniones de planificación
- Presentar especificaciones de seguridad
- Definir lineamientos y estándares
- Establecer criterios de aceptación de seguridad

12.2 Análisis de Requerimientos

Colaboración entre desarrollo y seguridad:

- Involucrar desarrollo en priorización de tareas de seguridad
- Evaluar impacto de seguridad de requerimientos
- Identificar componentes críticos
- Definir requerimientos de cumplimiento

BONUS

Framework de Requerimientos de Seguridad:

Clasificar requerimientos en categorías:

Autenticación y Autorización:

- Sistema de autenticación (SSO, MFA)
- Modelo de permisos (RBAC, ABAC)
- Gestión de sesiones

Datos:

- Cifrado en tránsito y en reposo
- Clasificación de información
- Retención y eliminación

Infraestructura:

- Segmentación de red
- Hardening de sistemas
- Gestión de parches

Logging y Monitoreo:

- Eventos a registrar
- Retención de logs
- Alertas y respuesta

12.3 Diseño

Participación de seguridad en arquitectura:

- Revisión de diseño de arquitectura
- Identificar sistemas involucrados
- Definir servicios a crear
- Modelar interacción de usuarios
- Aplicar principios de seguridad (defensa en profundidad, segregación)

13 DevSecOps: Codificación, Testing y Despliegue

13.1 Codificación

13.1.1 Static Application Security Testing (SAST)

Implementar análisis estático en pipeline:

- Integración en IDE (desarrollo local)
- Pre-commit hooks en Git
- Análisis en pull requests
- Bloqueo de merge ante vulnerabilidades críticas

Herramientas: SonarQube, Checkmarx, Sengrep, CodeQL

13.1.2 Revisión y Aprobación de Código

Proceso de peer review:

- Al menos un revisor técnico
- Checklist de seguridad
- Verificar ausencia de secretos hardcodeados
- Validar manejo de errores

13.1.3 Software Composition Analysis (SCA)

Seguridad en dependencias de código abierto:

- Identificar componentes con vulnerabilidades conocidas
- Análisis de licencias
- Monitoreo continuo de nuevas vulnerabilidades
- Políticas de actualización de dependencias

Herramientas: Snyk, Dependabot, OWASP Dependency-Check, WhiteSource

BONUS

Gestión de Deuda de Seguridad:

Priorizar según:

- **CVSS Score:** Severidad técnica (0-10)
- **Explotabilidad:** ¿Existe exploit público?
- **Exposición:** ¿El componente está expuesto externamente?
- **Impacto al negocio:** ¿Qué datos/funciones están en riesgo?

Matriz de decisión:

- Crítico + Expuesto = Fix inmediato
- Alto + Expuesto = Fix en sprint actual
- Medio + Interno = Planificar para próximo quarter
- Bajo + Interno = Backlog

13.2 Testing y Despliegue

13.2.1 Dynamic Application Security Testing (DAST)

Análisis dinámico de aplicaciones en ejecución:

- Escaneo en ambientes de staging
- Simulación de ataques (SQL injection, XSS, etc.)
- Testing de APIs
- Validación de configuraciones de seguridad

Herramientas: OWASP ZAP, Burp Suite, Acunetix, Netsparker

13.2.2 Seguridad en Contenedores

Búsqueda y análisis de vulnerabilidades:

- Escaneo de imágenes Docker
- Análisis de configuraciones de Kubernetes
- Políticas de admisión
- Detección de secretos en imágenes
- Uso de imágenes base mínimas (distroless)

13.2.3 Ambientes Diferenciados

Separación clara entre:

- **Desarrollo:** Datos sintéticos, controles relajados
- **Pruebas:** Datos anonimizados, testing exhaustivo
- **Staging:** Réplica de producción, últimas validaciones
- **Producción:** Datos reales, máxima seguridad

Principios:

- Sin datos de producción en ambientes inferiores
- Credenciales únicas por ambiente
- Segregación de red
- Configuraciones específicas por ambiente

BONUS**Interactive Application Security Testing (IAST):**

IAST combina ventajas de SAST y DAST:

Ventajas:

- Análisis en tiempo real durante testing
- Menor tasa de falsos positivos
- Identificación precisa de vulnerabilidades
- Trace de flujo de datos completo

Consideraciones:

- Requiere instrumentación de la aplicación
- Overhead de performance en testing
- No reemplaza SAST/DAST, complementa

Herramientas: Contrast Security, Hdiv Security

14 DevSecOps: Mantenimiento

14.1 Monitoreo

14.1.1 Herramientas de Telemetría

Implementar observabilidad completa:

- Logs centralizados (ELK Stack, Splunk)
- Métricas de aplicación (Prometheus, Grafana)
- Tracing distribuido (Jaeger, Zipkin)
- Monitoreo de integridad de archivos
- Detección de anomalías con ML

14.1.2 Web Application Firewall (WAF)

Protección en capa de aplicación:

- Filtrado de tráfico malicioso
- Protección contra OWASP Top 10
- Rate limiting
- Geo-blocking
- Virtual patching de vulnerabilidades conocidas

14.2 Reportería

Análisis de logs en tiempo real:

- Correlación de eventos
- Dashboards ejecutivos
- Métricas de seguridad (MTTR, MTDD)
- Tendencias y patrones
- Reportes de cumplimiento

14.3 Alertas

Generación de alertas relevantes y oportunas:

- Definir umbrales apropiados
- Reducir falsos positivos
- Clasificación por severidad
- Routing a equipos apropiados
- Playbooks automatizados

BONUS

SIEM (Security Information and Event Management):

Un SIEM centraliza logs y eventos para análisis de seguridad:

Capacidades clave:

- Agregación de logs de múltiples fuentes
- Correlación en tiempo real
- Detección de amenazas con reglas y ML
- Investigación forense
- Cumplimiento y auditoría

Soluciones populares:

- Splunk Enterprise Security
- IBM QRadar
- Azure Sentinel
- Google Chronicle
- Open source: Wazuh, OSSEC

Métricas importantes:

- Mean Time to Detect (MTTD)
- Mean Time to Respond (MTTR)
- Eventos por segundo procesados
- False positive rate

14.4 Bug Bounty

Externalizar búsqueda de vulnerabilidades:

- Comunidad de investigadores de seguridad
- Ampliación del alcance de testing
- Pago por vulnerabilidades encontradas
- Divulgación responsable

Plataformas: HackerOne, Bugcrowd, Synack, Intigriti

Consideraciones:

- Definir scope claramente
- Establecer reglas de engagement
- Tiempos de respuesta SLA
- Escalas de recompensas

15 Protocolo de Respuesta a Incidentes

15.1 Equipos de Respuesta Especializados

Estructura organizacional:

- **Incident Commander:** Coordina respuesta
- **Security Analyst:** Investiga y contiene
- **Communications Lead:** Maneja comunicación interna/externa
- **Subject Matter Expert:** Conocimiento técnico específico

Sistema de guardia (on-call):

- Rotaciones semanales o mensuales
- Alternativas y escalaciones
- Compensación por disponibilidad
- Runbooks y documentación

15.2 Invocar Actores Involucrados

Respuesta colaborativa:

- Equipo de desarrollo
- Operaciones/infraestructura
- Legal y cumplimiento
- Relaciones públicas
- Ejecutivos (según severidad)

15.3 Identificar Severidad del Incidente

15.3.1 Superficie de Ataque y Exposición

Evaluar:

- ¿Sistemas afectados son críticos?
- ¿Datos sensibles en riesgo?
- ¿Exposición externa o interna?
- ¿Número de usuarios/clientes impactados?

15.3.2 Establecer Prioridad

Sistema de clasificación:

- **P0/Crítico:** Brecha activa, datos expuestos, servicio caído
- **P1/Alto:** Vulnerabilidad explotable, riesgo inminente
- **P2/Medio:** Debilidad de seguridad, no explotable aún
- **P3/Bajo:** Mejora de seguridad, sin riesgo inmediato

15.3.3 CVSS (Common Vulnerability Scoring System)

Sistema estandarizado de puntuación (0-10):

Métricas base:

- Vector de ataque (red, adyacente, local, físico)
- Complejidad del ataque
- Privilegios requeridos
- Interacción del usuario
- Impacto (confidencialidad, integridad, disponibilidad)

Calculadora: <https://www.first.org/cvss/calculator/4.0>

BONUS

Fases del Incident Response según NIST SP 800-61:

1. Preparación

- Establecer políticas y procedimientos
- Formar equipo de respuesta
- Adquirir herramientas
- Realizar simulacros

2. Detección y Análisis

- Identificar indicadores de compromiso
- Determinar alcance
- Documentar evidencia
- Notificar stakeholders

3. Contención, Erradicación y Recuperación

- Aislar sistemas comprometidos
- Eliminar amenaza
- Restaurar operaciones
- Validar integridad

4. Actividad Post-Incidente

- Análisis de causa raíz
- Lecciones aprendidas
- Actualizar procedimientos
- Compartir inteligencia

15.4 Identificar Causa y Soluciones

Análisis técnico:

- Causa raíz del incidente
- Vector de ataque utilizado
- Controles que fallaron
- Indicadores de compromiso (IoCs)

Soluciones:

- **Inmediata:** Contención y mitigación temporal
- **Corto plazo:** Parche o configuración
- **Mediano/largo plazo:** Cambios arquitectónicos, mejoras de proceso

15.5 Documentar y Comunicar

15.5.1 Template de Documento de Incidente

Secciones esenciales:

- ID único del incidente
- Fecha y hora de detección
- Severidad y clasificación
- Sistemas y datos afectados
- Cronología de eventos
- Acciones tomadas
- Estado actual
- Próximos pasos
- Contactos clave

15.5.2 Comprometer Tareas y Asignar Responsabilidades

Gestión de acciones:

- Owner claramente asignado
- Fecha límite realista
- Prioridad definida
- Dependencias identificadas
- Seguimiento en herramienta centralizada

15.5.3 Comunicación Responsable

A partes afectadas:

- Clientes/usuarios afectados
- Reguladores (según aplique)
- Socios comerciales
- Transparencia apropiada
- Actualizaciones regulares

15.6 Reuniones de Equipos y Revisión

Durante el incidente:

- Standups frecuentes (cada 2-4 horas)
- Actualización de estado
- Coordinación de acciones
- Decisiones documentadas

Post-incidente:

- Retrospectiva sin culpa (blameless postmortem)
- Participación de todos los involucrados
- Identificación de mejoras
- Plan de acción correctivo

15.7 Post Mortem

Análisis estructurado:

- **Qué salió bien:** Controles que funcionaron, respuestas efectivas
- **Qué salió mal:** Fallas, demoras, confusión
- **En qué tuvimos suerte:** Factores fortuitos que limitaron daño
- **Aprendizajes:** Lecciones clave
- **Acciones de mejora:** Cambios concretos a implementar

Principios del post mortem:

- Sin culpa individual
- Enfoque en procesos y sistemas
- Documentación completa y pública (interna)
- Seguimiento de acciones comprometidas

BONUS

Cyber Kill Chain:

Modelo para entender etapas de ataques avanzados (APT):

1. **Reconnaissance:** Investigación del objetivo
2. **Weaponization:** Creación de payload malicioso
3. **Delivery:** Entrega del exploit
4. **Exploitation:** Ejecución del exploit
5. **Installation:** Instalación de backdoor
6. **Command & Control:** Comunicación con atacante
7. **Actions on Objectives:** Robo de datos, destrucción

Defensa en profundidad: Interrumpir la cadena en cualquier etapa detiene el ataque.

16 Metodologías Populares para SSDLC

16.1 Microsoft Security Development Lifecycle (SDL)

Framework de Microsoft usado desde Windows Vista:

Fases principales:

- Capacitación en seguridad
- Requerimientos de seguridad
- Modelado de amenazas
- Estándares de diseño
- Herramientas de análisis
- Pruebas de seguridad
- Plan de respuesta a incidentes

Recursos: <https://www.microsoft.com/en-us/securityengineering/sdl>

16.2 OWASP Software Assurance Maturity Model (SAMM)

Framework abierto para evaluar y mejorar seguridad:

Dominios de negocio:

- **Governance:** Strategy, Policy, Education
- **Design:** Threat Assessment, Security Requirements, Architecture
- **Implementation:** Secure Build, Deployment, Defect Management
- **Verification:** Architecture Assessment, Requirements Testing, Security Testing
- **Operations:** Incident Management, Environment Management, Operational Enablement

Niveles de madurez (0-3) para cada práctica.

Recursos: <https://owasp.org/www-project-samm/>

16.3 NIST Cybersecurity Framework (NIST CSF)

Framework voluntario para gestión de riesgos:

Cinco funciones principales:

- **Identify:** Activos, riesgos, estrategia
- **Protect:** Controles preventivos
- **Detect:** Monitoreo y detección
- **Respond:** Plan de respuesta
- **Recover:** Recuperación y mejora

Recursos: <https://www.nist.gov/cyberframework>

16.4 Google Supply-chain Levels for Software Artifacts (SLSA)

Framework para integridad de la cadena de suministro:

Niveles (1-4):

- **SLSA 1:** Documentación del proceso de build
- **SLSA 2:** Control de versiones + servicio de build
- **SLSA 3:** Auditabilidad de código fuente y build
- **SLSA 4:** Build hermético y revisión de dos personas

Recursos: <https://slsa.dev>

BONUS

Comparación de frameworks:

Microsoft SDL:

- Mejor para: Organizaciones maduras, productos comerciales
- Ventaja: Detallado y prescriptivo
- Desventaja: Puede ser pesado para startups

OWASP SAMM:

- Mejor para: Evaluar madurez actual, mejora incremental
- Ventaja: Flexible, agnóstico de tecnología
- Desventaja: Requiere interpretación

NIST CSF:

- Mejor para: Cumplimiento, comunicación ejecutiva
- Ventaja: Ampliamente reconocido, orientado a riesgo
- Desventaja: Alto nivel, necesita complementos

SLSA:

- Mejor para: Proyectos open source, supply chain
- Ventaja: Enfoque moderno en CI/CD
- Desventaja: Relativamente nuevo

Recomendación: No son mutuamente excluyentes. Usar NIST CSF para estrategia, SAMM para evaluación de madurez, SDL/SLSA para prácticas específicas.

16.5 ISO/IEC 27001

Estándar internacional para gestión de seguridad de información:

Características:

- Certificación formal
- Sistema de gestión (SGSI)
- Enfoque en procesos
- Mejora continua (PDCA)
- Controles (Anexo A)

Recursos: <https://www.iso.org/standard/27001>

17 Referencias y Recursos

17.1 OWASP Top 10

Lista actualizada de riesgos más críticos en aplicaciones web:

Top 10 (2021):

1. Broken Access Control
2. Cryptographic Failures
3. Injection
4. Insecure Design
5. Security Misconfiguration
6. Vulnerable and Outdated Components
7. Identification and Authentication Failures
8. Software and Data Integrity Failures
9. Security Logging and Monitoring Failures
10. Server-Side Request Forgery (SSRF)

Recurso: <https://owasp.org/www-project-top-ten/>

BONUS

OWASP ASVS (Application Security Verification Standard):

Complemento del Top 10, más detallado:

- 3 niveles de verificación (1: básico, 2: estándar, 3: avanzado)
- 14 categorías de requerimientos
- Checklist concreto para testing
- Base para herramientas automatizadas

Úsalo como guía para:

- Definir requerimientos de seguridad
- Evaluación de arquitectura
- Criterios de aceptación en QA
- RFPs para selección de vendors

Recurso: <https://owasp.org/www-project-application-security-verification-standard/>

17.2 Calculadora CVSS

Sistema estandarizado para calificar severidad de vulnerabilidades.

Versiones:

- CVSS 3.1: Actual estándar
- CVSS 4.0: Última versión (2023)

Recurso: <https://www.first.org/cvss/calculator/4.0>

17.3 Recursos de Aprendizaje

17.3.1 Secure Developer Podcast

Podcast sobre desarrollo seguro:

- Entrevistas con expertos
- Casos de estudio
- Mejores prácticas
- Herramientas y técnicas

Recurso: <https://www.devseccon.com/the-secure-developer-podcast/>

17.3.2 LiveOverflow en YouTube

Canal educativo de seguridad:

- CTF writeups
- Análisis de vulnerabilidades
- Técnicas de hacking
- Tutoriales prácticos

Recurso: <https://www.youtube.com/@LiveOverflow>

BONUS

Recursos adicionales recomendados:

Cursos y Certificaciones:

- SANS SEC522: Application Security
- OSWE: Offensive Security Web Expert
- AWS/Azure Security Certifications
- PortSwigger Web Security Academy (gratuito)

Plataformas de práctica:

- HackTheBox: Labs y challenges
- TryHackMe: Guiado, para principiantes
- PentesterLab: Enfocado en web
- WebGoat/DVWA: Aplicaciones vulnerables

Libros fundamentales:

- *The Web Application Hacker's Handbook*
- *Secure by Design*
- *Building Secure and Reliable Systems* (Google)
- *Threat Modeling: Designing for Security*

18 Conclusiones

La seguridad en el SDLC no es un checkpoint único, sino un proceso continuo integrado en cada etapa.

18.1 Principios Fundamentales

1. **Shift Left:** Detectar problemas temprano reduce costos exponencialmente
2. **DevSecOps:** Seguridad es responsabilidad compartida
3. **Automatización:** Herramientas integradas permiten escalar sin sacrificar velocidad
4. **Defensa en profundidad:** Múltiples capas compensan fallas
5. **Mejora continua:** Aprender de incidentes constantemente

18.2 Cambio Cultural

El éxito depende más de personas y procesos que de herramientas:

- Fomentar mentalidad de seguridad en desarrolladores
- Reconocer y recompensar prácticas seguras
- Crear seguridad psicológica para reportar problemas
- Invertir en capacitación continua
- Colaboración entre equipos

18.3 Balance

La seguridad perfecta es imposible y no deseable:

- Priorizar basándose en riesgo real
- No sacrificar velocidad innecesariamente
- Aceptar riesgos de forma consciente
- Medir efectividad regularmente
- Adaptar según contexto

BONUS

Métricas para un SDLC Seguro:

Indicadores de efectividad:

- % vulnerabilidades detectadas antes de producción
- Tiempo promedio de remediación
- Cobertura de análisis SAST/DAST/SCA
- % de código revisado por pares
- Incidentes de seguridad en producción

Indicadores de madurez:

- % desarrolladores con capacitación
- Tiempo de ciclo (commit a producción)
- Frecuencia de despliegues
- % servicios con modelado de amenazas
- Tasa de adopción de herramientas

Meta: Seguridad como habilitador del negocio, no obstáculo. DevSecOps permite mayor velocidad y mejor seguridad simultáneamente.

18.4 Próximos Pasos

Para implementar un SDLC seguro:

1. **Evaluar estado actual:** Usar SAMM para baseline
2. **Priorizar mejoras:** Quick wins de alto impacto
3. **Obtener buy-in:** Demostrar valor al negocio
4. **Empezar pequeño:** Piloto con un equipo, iterar, escalar
5. **Automatizar progresivamente:** Integrar herramientas
6. **Medir y comunicar:** Reportar métricas regularmente
7. **Iterar:** Ajustar basándose en feedback

La seguridad es un viaje, no un destino.

Material del curso IIC2531 - Seguridad Computacional

Incluye charla invitada de:
Carlos Oliva - Security Architect @ ZeroFox
carlos.oliva@gmail.com